

✓ Introduzione

1) Quali sono le principali differenze nelle esigenze che intercorrono quando si programma un software di piccole dimensioni realizzato per soddisfare una esigenza limitata e un software complesso?

Quando si lavora su un software di dimensioni limitate con gruppi ristretti da massimo un paio di persone non è essenziale istituire forme rigorose di organizzazione di processo. Infatti, la comunicazione può essere assente, il lavoro non ha necessità di essere coordinato e spesso pochi documenti e materiali sono passati tra i componenti del gruppo.

Nel caso opposto invece è necessario stabilire accordi su più aspetti che prima risultavano trascurabili come, ad esempio, il linguaggio di programmazione, le tecnologie scelte per l'implementazione, l'ambiente di sviluppo condiviso ed il sistema di controllo di versione. In più è importante suddividere tramite il metodo "Divide Et Impera" le funzionalità da implementare, in modo da ottenere una serie di compiti più facilmente assegnabili ai componenti della squadra. Più il progetto cresce di dimensioni, più diventa inoltre complicato stimare lo sforzo necessario per sviluppare tutte le funzionalità richieste dal committente.

2) Che cosa si intende per ingegneria del software e quali sono i problemi che questa disciplina si propone di affrontare?

Non esiste una definizione comunemente accordata per il termine ingegneria del software, tuttavia ne esistono varie versioni. Ad esempio, può essere considerata come una disciplina dell'ingegneria che riguarda tutti gli aspetti dello sviluppo del software dai requisiti fino al mantenimento futuro. La definizione data dal CMU la definisce una forma di ingegneria capace di impiegare i principi dell'IT per ottenere una costa efficace dal punto di vista dei costi; lo standard IEEE 610 ne parla come un approccio sistematico, disciplinato e quantificabile allo sviluppo e al mantenimento. Questa disciplina si pone l'obiettivo di regolamentare lo sviluppo del prodotto (software) per risolvere i problemi nati dalla "crisi del software", dovuta ad un numero elevato di progetti che fallivano per mancate deadline o budget superati.

3) Che cosa implica la produzione di un software e qual è l'output generalmente atteso da questo processo

La produzione di un software implica un processo articolato in più fasi:

Inizialmente con la fase di specifica dei **requisiti** vengono raccolte le richieste del cliente, gli input e gli output attesi, tramite diagrammi UML e altri strumenti applicativi. In questa fase vengono suddivisi in requisiti funzionali, ovvero legati al comportamento del programma e non funzionali, legati invece al design e alle prestazioni necessarie.

L'iter procede con la fase di **progettazione**, all'interno della quale vengono definiti gli stili architetturali, effettuata la decomposizione funzionale e stabilite le metriche di valutazione. Gli strumenti applicativi utilizzati sono nuovamente gli UML uniti ai design pattern.

Si giunge poi alla fase **implementativa** nella quale, in base alla completezza del progetto, alterno fasi di design e fasi di scrittura effettiva del codice.

All'interno di quest'ultima inizia il processo di **test** e debug degli artefatti prodotti, definendo

dei casi di test per verificare il funzionamento del codice e, in caso di errori, risolverli tramite opportuno debugging. Distinguiamo pertanto testing white-box (con conoscenza estesa del codice) e black-box (legato al funzionamento del complesso).

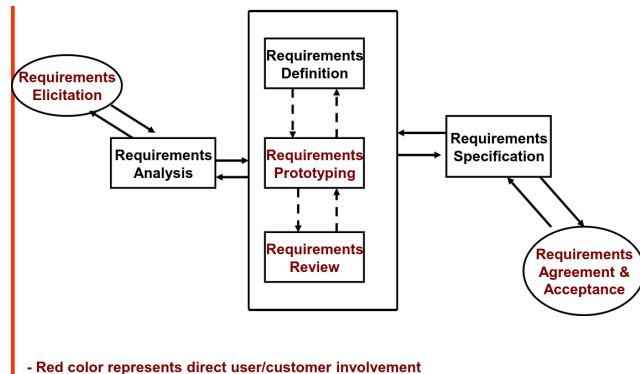
L'ultimo passaggio è il **rilascio** e la distribuzione del prodotto, che darà inizio ad un ulteriore processo di mantenimento e supporto dello stesso, qualora si tratti di un progetto più ampio.

In generale quindi l'output atteso non è unicamente l'eseguibile del codice rilasciato, ma anche una serie di artefatti accessori come manuali di utilizzo, documenti, diagrammi di flusso dei dati e di funzionamento oltre che servizi di formazione e istruzione sull'uso.

✓ Requirements Engineering → COMPLETA

1) Quali sono le principali fasi in cui il processo di specifica dei requisiti può essere suddiviso?

(Processo di analisi dei requisiti serve per raccogliere cosa deve fare il software. Requisiti funzionali: cosa deve fare. Requisiti non funzionali: come lo deve fare. Gestire i requisiti e coinvolgere l'utente è una cosa fondamentale. L'analisi dei requisiti è un processo che si compone di 7 punti. Ma ancora più importante è che va fatto prima è fare un piano in cui si decidono le persone coinvolte perché magari devo far fare qualcosa al cliente, si decide la metodologia da usare e il tempo che si vuole dedicare. Poi possiamo iniziare con i 7 punti).



Il processo di specifica dei requisiti, all'interno dell'ingegneria dei requisiti, può essere suddiviso nelle seguenti fasi principali: **Elicitazione dei requisiti** → Raccolta delle informazioni dagli utenti e dagli stakeholder tramite interviste, analisi documentale, osservazione, workshop. **Documentazione e definizione** → Formalizzazione preliminare delle informazioni raccolte in un linguaggio coerente e strutturato. **Analisi dei requisiti** → Verifica di completezza, coerenza, eliminazione di ambiguità, individuazione di dipendenze e priorità, clustering. **Prototipazione** → Realizzazione di prototipi, mockup o diagrammi che supportano la comprensione dei requisiti, specialmente delle interfacce utente. **Specifiche dei requisiti** → Produzione del documento formale SRS (Software Requirements Specification) o documenti equivalenti. **Revisione e convalida** → Verifica della correttezza dei requisiti tramite review, walkthrough, prototipi o incontri con il cliente. **Accordo e accettazione** → Il cliente e gli stakeholder accettano formalmente i requisiti, costituendo la "baseline" del progetto.

Non tutte queste attività sono necessarie nella stessa misura per tutti i progetti software. Quanto e quando queste attività vengono praticate e in quale sequenza è il tema centrale del processo dei requisiti. Specifiche dei requisiti → stabiliscono la baseline per il progetto. Qualsiasi richiesta di modifica deve essere gestita attentamente attraverso un processo di controllo delle modifiche per evitare modifiche incontrollate. L'errore di non dedicare tempo e impegno alla raccolta e alla comprensione dei requisiti può essere costoso. È molto difficile gestire la pianificazione e i costi del progetto senza requisiti chiari e documentati. All'altro estremo dello spettro ci sono i costi eccessivi degli sforzi dedicati a prototipi, revisioni e creazione di documenti voluminosi.

2) Qual è la differenza tra l'operazione di elicitation eseguita a alto livello / a livello di dettaglio (riguardare)

Livello alto: comprendere la logica aziendale e la giustificazione del software.

Livello di dettaglio: raccogliere informazioni dettagliate sui bisogni e i desideri degli utenti.

Gli analisti devono preparare una serie strutturata di domande per gli utenti. I metodi di elicitazione possono includere: Verbale (interviste, riunioni), Scritto (moduli preformattati), Moduli online.

Il contatto diretto con gli utenti spesso innesca utili domande di follow-up e consente agli utenti di ampliare il loro contributo.

Gli analisti dovrebbero raccogliere le informazioni esistenti da documenti già presenti in azienda.

ALTO LIVELLO → L'analista dei requisiti deve interagire con il management e i dirigenti per comprendere le motivazioni aziendali. Le motivazioni aziendali vengono tradotte in requisiti, espressi come vincoli sul prodotto software e sul progetto. Le categorie chiave di informazioni per il profilo aziendale di alto livello includono: Opportunità / Esigenze, Ambito, Funzionalità principali, Fattori di successo, Costo e Tempo (vincoli principali).

Il progetto è spesso considerato di successo se gli obiettivi aziendali di alto livello vengono raggiunti.

Caratteristiche dell'utente: comprendere gli utenti quotidiani è essenziale per l'adozione del sistema e una formazione adeguata. I dirigenti potrebbero non essere utenti quotidiani, ma il successo del progetto dipende dalla formazione e dal supporto degli utenti e dall'adattamento del sistema alle loro esigenze.

REQUISITI DETTAGLIATI → Gli utenti spesso iniziano a parlare di possibili soluzioni, soprattutto se esiste già un sistema simile. Durante queste sessioni, sia gli analisti che gli utenti possono esplorare idee creative e tecniche. La funzionalità individuale è spesso il punto di partenza per l'individuazione dei requisiti. Gli analisti chiedono agli utenti informazioni sui loro problemi in termini di funzioni da eseguire. È necessario raccogliere dati e formati. Dati di input e output dell'applicazione. Interfacce utente. Trasferimento dei dati. Errori e messaggi con i relativi metodi di ripristino e ripetizione dell'operazione...

Affidabilità, prestazioni, sicurezza e adattabilità: suggerimenti chiave per i requisiti non funzionali. Altri vincoli possono includere portabilità e manutenibilità. I requisiti relativi al progetto possono richiedere l'uso di uno strumento o di un linguaggio di programmazione specifico.

3) Indicare le principali caratteristiche dell'analisi dei requisiti di alto livello e di dettaglio (uguale a quella sopra ?)

4) Cosa si intende per analisi dei requisiti e in che modo si differisce dall'operazione di elicitation

Elicitazione: È il processo di raccolta delle informazioni dai vari stakeholder.

Risponde alla domanda: "Quali sono i requisiti?"

Analisi dei requisiti: È il processo di elaborazione, classificazione, validazione e organizzazione di ciò che è stato raccolto.

Risponde alla domanda: "Questi requisiti sono completi, coerenti, non ambigui, prioritizzati e realizzabili?"

Una volta raccolte le informazioni, vengono identificati i confini del sistema. L'identificazione dei confini del sistema inizia con la delineazione di ciò che è incluso ed escluso dal sistema ma che potrebbe comunque essere necessario per l'interfaccia con il sistema.

Nella metodologia dei casi d'uso OO, i passaggi per l'identificazione dei seguenti fattori costituiscono una buona metodologia di analisi dei requisiti: Attori, Casi d'uso correlati, Condizioni al contorno.

In sintesi:

Elicitazione = raccolta delle informazioni

Analisi = interpretazione, strutturazione e verifica delle informazioni → va in aiuto UML

5) La prototipazione dei requisiti include anche la realizzazione di diagrammi UML di tipo Use Case. Che cosa si vuole rappresentare con questo tipo di diagrammi? Quali sono le informazioni che rappresentano questo tipo di diagrammi.

Scopo: Catturare i requisiti funzionali di un sistema.

Approccio: Descrivere le interazioni tra gli utenti (attori) e il sistema, come una narrazione di come il sistema viene utilizzato. Utilizziamo un diagramma dei casi d'uso per rispondere alle seguenti domande:

1. Cosa viene descritto? → Il sistema
2. Chi interagisce con il sistema? → Gli attori.
3. Cosa possono fare gli attori? → I casi d'uso

Il diagramma dei casi d'uso è essenziale per una progettazione tecnica dettagliata. Esprime le aspettative del cliente nei confronti del sistema. Il diagramma documenta i requisiti che il sistema deve soddisfare.

Se i casi d'uso vengono dimenticati o specificati in modo errato, le conseguenze possono essere estremamente gravi: aumentano i costi di sviluppo e manutenzione, gli utenti sono insoddisfatti.

Questo è uno scenario: una sequenza di passaggi che descrive l'interazione tra l'utente e un sistema. Questa è una cosa che può accadere. (esempio caso d'uso su slide 3)

Passaggi chiave:

- 1) Identificazione degli attori
- 2) Identificazione degli scenari
- 3) Identificazione dei casi d'uso

Funzionalità: Rappresentano gli elementi funzionali di un sistema. Utile per pianificare progetti iterativi, in cui ogni iterazione fornisce un set di funzionalità.

Casi d'uso: Forniscono una descrizione di come gli attori interagiscono con il sistema.

Concentrarsi su "chi fa cosa" e perché, piuttosto che su piccoli dettagli implementativi.

Relazione tra funzionalità e casi d'uso.

Funzionalità e casi d'uso sono strettamente correlati: Una funzionalità può corrispondere a: Un caso d'uso completo. Uno scenario all'interno di un caso d'uso. Un passaggio in un caso d'uso.

CHAT: Cosa si vuole rappresentare con i diagrammi Use Case?

1. Attori: Gli attori sono entità che interagiscono con il sistema. Possono essere persone (es. utenti finali, amministratori), altri sistemi o dispositivi esterni. Ogni attore rappresenta un ruolo che può svolgere un determinato insieme di operazioni sul sistema.

2. Use Case (Casi d'uso): Un use case è una sequenza di azioni che il sistema esegue in risposta a un'azione dell'attore. Ogni use case descrive una funzionalità specifica che il sistema deve offrire agli attori per soddisfare i loro obiettivi. Ad esempio, "Login utente" o "Registrazione di un nuovo prodotto".
3. Relazioni: I diagrammi Use Case mostrano come gli attori interagiscono con i vari use case. Le relazioni più comuni sono:
 - a. Associazione (una linea semplice che collega un attore a un use case): indica che l'attore può eseguire un determinato use case.
 - b. Inclusione (include): uno use case può includere un altro che rappresenta una parte sempre necessaria dell'azione.
 - c. Estensione (extends): uno use case può essere esteso con funzionalità opzionali in determinate condizioni.
4. Confini del sistema: Il diagramma aiuta a delimitare il perimetro del sistema, evidenziando cosa è incluso e cosa no. Gli use case rappresentano ciò che il sistema farà, mentre ciò che è fuori dal sistema o non incluso nel suo comportamento non viene rappresentato.

Quali informazioni rappresentano i diagrammi Use Case? Un diagramma Use Case include principalmente le seguenti informazioni:

1. Attori: Sono rappresentati da figure stilizzate, generalmente disegnate come icone di piccole persone o etichette che descrivono i loro ruoli (es. "Utente", "Amministratore", "Sistema esterno", ecc.).
2. Use Case: Rappresentati come ovali, ognuno descrive un'operazione o una funzionalità del sistema (es. "Effettuare login", "Registrazione un ordine", "Visualizzare report", ecc.).
3. Relazioni tra attori e use case: Linee che collegano gli attori agli use case per indicare le interazioni. Ogni relazione rappresenta un'azione o un obiettivo che l'attore cerca di raggiungere con l'interazione.
4. Relazioni tra i casi d'uso:
 - a. Inclusione: Quando un use case obbligatoriamente chiama un altro use case (es. "Autenticazione" potrebbe essere un use case incluso nel caso "Login").
 - b. Estensione: Quando un use case opzionalmente estende un altro caso d'uso (es. "Pagamento con carta di credito" potrebbe estendere un caso d'uso di "Pagamento" in alcune condizioni).

In sintesi: I diagrammi Use Case si concentra sull'identificazione delle funzionalità principali del sistema e sulle interazioni tra gli utenti e il sistema. Sono utili per capire i requisiti funzionali a un livello alto, senza entrare nei dettagli tecnici. Questi diagrammi sono fondamentali per chiarire cosa deve fare il sistema in risposta alle azioni degli attori e per comunicare questi requisiti agli sviluppatori, agli analisti e agli altri stakeholder.

6) Introduzione di un problema che deve essere descritto dallo studente con un diagramma UML di tipo Use Case

Si descriva in forma testuale e con il corrispondente formalismo grafico il processo di acquisto di una bevanda dal distributore automatico. L'utente può decidere di pagare o utilizzando monete o utilizzando una chiavetta. Si rappresenti il caso d'uso utilizzando un elenco di passi e identificando anche possibili estensioni per gestire situazioni particolari (il prodotto non è disponibile, l'importo non è sufficiente, le monete non sono conformi ...).

Immagina un sistema di e-commerce in cui gli utenti possono fare acquisti, aggiungere articoli al carrello, registrarsi, e pagare per gli acquisti. Ogni attività viene rappresentata da un caso d'uso specifico.

7) Quali sono gli standard con cui si rappresentano i requisiti?

Dopo essere stati analizzati e revisionati, i requisiti dovrebbero essere documentati in un documento di specifiche dei requisiti. Il livello di dettaglio dipende da:

- Dimensioni e complessità del progetto
- Numerose release successive pianificate
- Attività di supporto clienti previste
- Conoscenza ed esperienza degli sviluppatori nel dominio

Una specifica dettagliata è essenziale per un'ampia base di utenti, al fine di consentire manutenzione e supporto organizzati. Se sviluppatori, tester e progettisti hanno una conoscenza limitata del dominio una specifica dettagliata è essenziale.

ISO / IEC / IEEE

Questo documento

- specifica i processi richiesti implementati nelle attività di ingegneria che danno origine ai requisiti per sistemi e prodotti software;
- fornisce linee guida per l'applicazione dei requisiti e dei processi correlati ai requisiti;
- specifica le informazioni richieste prodotte attraverso l'implementazione dei processi dei requisiti;
- specifica il contenuto richiesto delle informazioni richieste;
- fornisce linee guida per il formato delle informazioni richieste e correlate.

Il processo dei requisiti deve produrre i seguenti elementi informativi:

- 1) Specifica dei requisiti aziendali (BRS); (vedi sezioni slide)
- 2) Specifica dei requisiti degli stakeholder (StRS);
- 3) Specifica dei requisiti di sistema (SyRS);
- 4) Specifica dei requisiti software (SRS) --> più importante

Scopo: Ambito, Prospettiva del prodotto, Funzioni del prodotto, Limitazioni, Presupposti e dipendenze, Ripartizione dei requisiti. Requisiti specificati. Descrivono ogni input (stimolo) nel sistema software, ogni output (risposta) dal sistema software e tutte le funzioni eseguite dal sistema software in risposta a un input o a supporto di un output. Interfacce esterne. Requisiti di usabilità. Requisiti di prestazioni. Requisiti logici del database. Conformità agli standard. Attributi del sistema software --> viene rappresentato con una tabella

✓ Design Pattern → COMPLETA

1) Quali sono i principi di programmazione che motivano la realizzazione e l'uso di design pattern

I design pattern ci permettono di applicare in maniera ancora più estesa i principi SOLID. in particolare:

- forniscono un insieme di soluzioni comuni a problemi di design noti
- permettono di avere un linguaggio strutturale comune tra più sviluppatori
- permettono di velocizzare il processo di sviluppo tramite l'utilizzo (se tutti li usano sono vantaggiosi, anche per coerenza)

Utilizzando i design pattern possiamo creare livelli ancora più sofisticati di astrazione, in modo da rendere il nostro codice completamente modulare e flessibile ai cambiamenti

2) Viene fornito uno scenario e viene richiesto di selezionare il pattern opportuno (giustificando la scelta) e di descrivere le classi principali anche attraverso il codice.

Non richiede risposta (ci servirebbe lo scenario)

3) Per ogni pattern introdotto a lezione (Strategy, Observer, Decorator, Factory, Singleton, Command, Adapter, Facade, Template, Iterator, Composite, State, Proxy, Builder, Bridge, Chain of Responsibility) può essere chiesto:

- a) Motivazione / Uso / Funzionamento / Esempio rilevante / PseudoCodice / Applicazione a un determinato ambito
- b) Class Diagram
- c) In che modo il diagramma soddisfa un principio di programmazione

Strategy:

a) Motivazione: Evitare problemi di ereditarietà rigida quando le classi derivate hanno comportamenti diversi che cambiano spesso. **Uso:** Definire una famiglia di algoritmi, incapsularli e renderli intercambiabili. **Funzionamento:** Si delega il comportamento a un oggetto "composto" invece di ereditarlo.

Esempio rilevante: Un gioco di ruolo con personaggi come King o Knight che usano armi diverse (Knife, Axe..) → Si crea un'interfaccia chiamata FightBehavior con un singolo metodo fight() e poi si implementano diverse classi concrete varianti del comportamento (tipo AxeBehavior..).

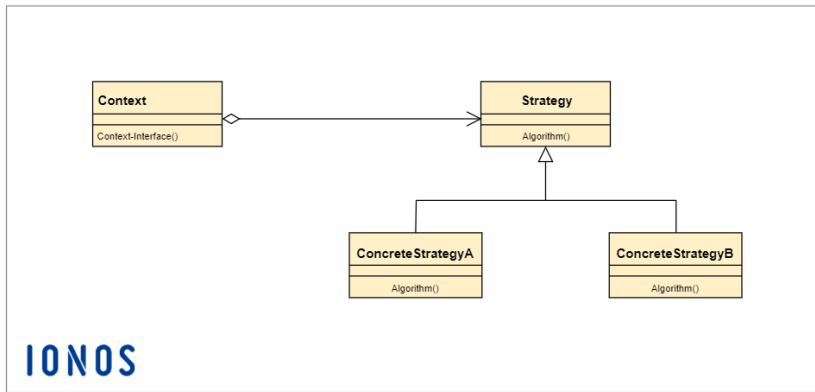
Vantaggio → Poiché il comportamento è un oggetto e non un metodo fisso, può essere cambiato mentre il gioco è in esecuzione.

Pseudo codice:

Applicazione a un determinato ambito: giochi o sistemi dove gli algoritmi variano per contesto.

b) Class Diagram: Il diagramma prevede una classe Context (es. Character) che mantiene un riferimento a un'interfaccia Strategy (es FightBehavior).

Diverse classi ConcreteStrategy implementano l'interfaccia con algoritmi specifici.



c) In che modo il diagramma soddisfa un principio di programmazione

l'incapsulare ciò che varia: isola i comportamenti di combattimento dal resto della classe Character.

favorire la composizione rispetto all'ereditarietà: Il personaggio ha un comportamento invece di "essere" il comportamento.

programmare verso un'interfaccia, non un'implementazione.

Observer

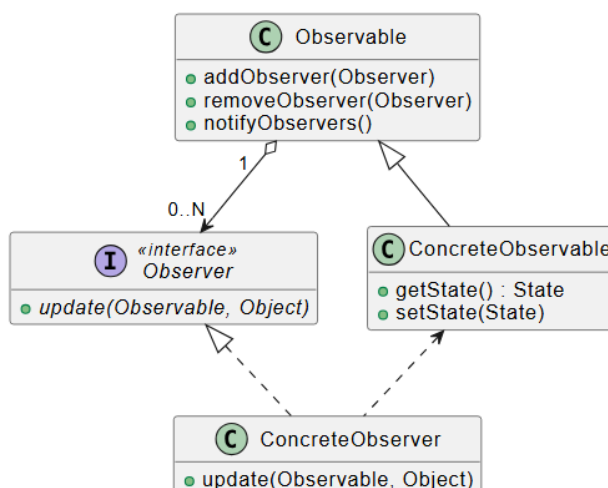
a) Motivazione: Mantenere gli oggetti informati automaticamente quando accade qualcosa a cui sono interessati. **Uso:** Definire una dipendenza uno-a-molti: se un oggetto cambia stato, i dipendenti vengono notificati. **Funzionamento:** E' un'architettura di tipo publisher-subscriber che definisce una dipendenza uno a molti tra oggetti. Esiste un Subject (Editore) che mantiene una lista di Observers (Abbonati) e li chiama tramite un metodo update().

Esempio rilevante: Una stazione meteo (WeatherData) che aggiorna vari display quando cambiano temperatura o umidità.

PseudoCodice:

Applicazione a un determinato ambito: Applicazioni con comunicazioni broadcast tra oggetti, o frameworks GUI (es Swing).

b) Class Diagram: La soluzione proposta dal pattern Observer è quella di estrarre la parte comune (lo stato) e isolarlo in un oggetto a parte, detto Subject: tale oggetto verrà osservato da tutte le viste, le cui classi prendono ora il nome di Observer. Si sta cioè centralizzando la gestione dello stato, quindi saranno presenti n classi che osserveranno una classe centrale reagiranno ad ogni cambiamento di stato di quest'ultima.



c) Principi di programmazione soddisfatti: Loose Coupling (Accoppiamento Debole): Il soggetto sa solo che l'osservatore implementa una certa interfaccia; non ha bisogno di conoscere le classi concrete dei display.

Decorator

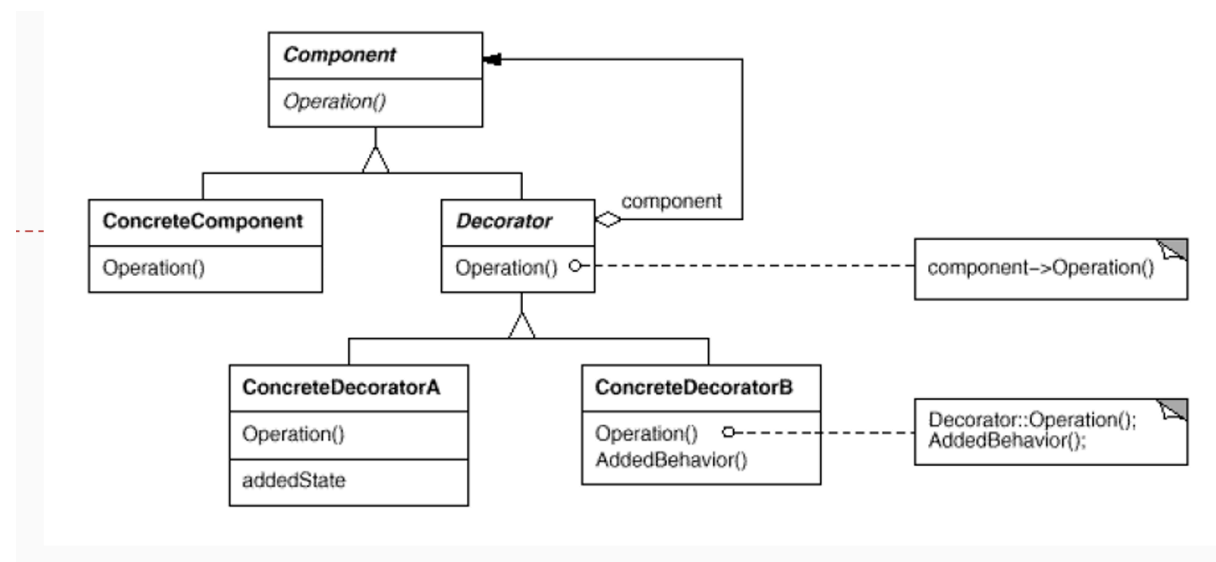
a) Motivazione: Evitare l'esplosione di sottoclassi quando si hanno tantissime combinazioni di caratteristiche opzionali. **Uso:** Aggiungere responsabilità a un oggetto in modo dinamico senza modificare il codice delle classi sottostanti. **Funzionamento:** Il decorator ha lo stesso supertipo dell'oggetto che decora e lo "avvolge" (wrapping). I Decorator ricevono come oggetto da ricoprire al momento della costruzione un generico Component, in quanto questo permette ai decorator di decorare oggetti già decorati. Questo approccio "ricorsivo" permette di creare una catena di decorator che definisca a runtime in modo semplice e pulito oggetti dotati di moltissime funzionalità aggiunte, così facendo si alleggerisce la fase di compiling, aggiungendo determinate funzionalità dinamicamente.

Esempio rilevante: Modellare con oggetti diverse pizze, variabili per base (es. normale, integrale, senza glutine) e ingredienti. Ogni pizza sarà un oggetto che implementa l'interfaccia comune Pizza, e il metodo toString() deve elencarne base e ingredienti.

PseudoCodice:

Applicazione a un determinato ambito: Le API Java I/O

b) Class Diagram: Include un Component astratto, ConcreteComponent, un Decorator (che estende Component e ha un riferimento a Component) e i ConcreteDecorator.



c) Principi di programmazione soddisfatti: Open-Closed Principle: Le classi devono essere aperte all'estensione ma chiuse alla modifica.

Factory

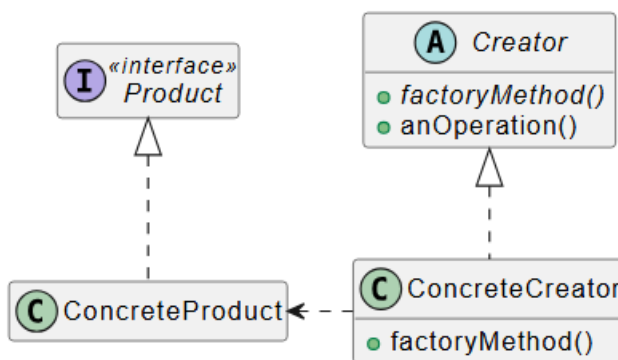
a) Motivazione: Disaccoppiare la creazione di un oggetto dalla classe che lo utilizza. **Uso:** Lasciare che siano le sottoclassi a decidere quale classe concreta istanziare

Funzionamento: Il creatore definisce un metodo astratto (factory method) che le sottoclassi implementano. Si definisce dunque un'interfaccia per creare un Product ma lascia al Creator concreto la scelta di cosa creare effettivamente: in questo modo all'interno della classe astratta Creator è possibile scrivere dei metodi che richiedono la creazione di un Product pur senza sapere di preciso il tipo dell'oggetto che verrà creato.

PseudoCodice:

Applicazione a un determinato ambito: Un PizzaStore con un metodo createPizza(). NaplesPizzaStore creerà pizze in stile Napoli, mentre MilanPizzaStore pizze in stile Milano. Oppure la gestione dell'apertura di un documento su Word (Si definisce un metodo astratto createDocument() all'interno di Application); ciò permette alla classe Application di gestire l'apertura di un file in modo generico, delegando alle sottoclassi l'onere di decidere quale classe specifica di documento istanziare.

b) Class Diagram: Esso definisce una classe astratta Creator dotata di metodi fabbrica astratti che restituisce un'istanza di un tipo aderente all'interfaccia Product a cui il Client è interessato: a quale classe appartenga effettivamente tale istanza (Product concreto) è però lasciato ad un Creator concreto tra i tanti che estendono la classe astratta; idealmente dovrebbe esserci un creatore concreto per ogni tipo di prodotto concreto che implementa l'interfaccia Product.



c) Principi di programmazione soddisfatti: Dependency Inversion Principle: Dipendere da astrazioni, non da classi concrete.

Singleton

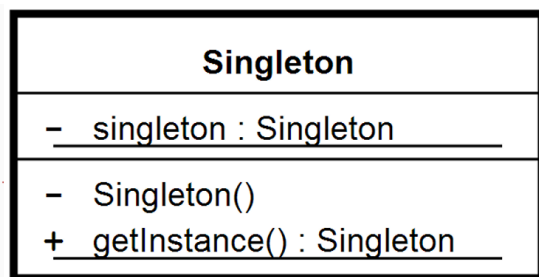
Motivazione: Esistono oggetti di cui è necessaria una sola istanza all'interno di un'applicazione per evitare conflitti o spreco di risorse, come pool di thread, cache o driver di dispositivi. **Uso:** Garantire che una classe abbia un'unica istanza e fornire un punto di accesso globale ad essa. **Funzionamento:** Si rende il costruttore privato per impedire l'istanziamento esterno e si utilizza un metodo statico che crea l'oggetto solo se non esiste già (lazy instantiation)

PseudoCodice:

Esempio rilevante: La gestione di una caldaia per la produzione di Parmigiano Reggiano, dove più istanze del controller potrebbero causare errori critici nella bollitura.

Applicazione in ambito determinato: Driver di schede grafiche, sistemi di logging o gestione delle impostazioni di registro.

b) Class Diagram: Esiste una classe unica che contiene una variabile statica `uniqueInstance` (di tipo `Singleton`), un costruttore privato e il metodo statico `getInstance()` che restituisce l'istanza.



c) Principi di programmazione soddisfatti: Il diagramma soddisfa il principio di Incapsulamento e Controllo dell'Accesso: ossia nascondendo il costruttore (`private`), la classe assume il controllo totale sulla propria istanziazione, impedendo a classi esterne di violare la regola dell'istanza unica.

Command

a) Motivazione: Disaccoppiare l'oggetto che effettua una richiesta da quello che sa come eseguirla. **Uso:** Incapsulare una richiesta come oggetto, permettendo di parametrizzare i client con diverse richieste, accodare o loggare operazioni e supportare l'undo (annullamento). **Funzionamento:** Un oggetto `Invoker` detiene un comando; quando viene attivato, chiama il metodo `execute()` del comando, il quale a sua volta invoca le azioni necessarie sul `Receiver`.

PseudoCodice:

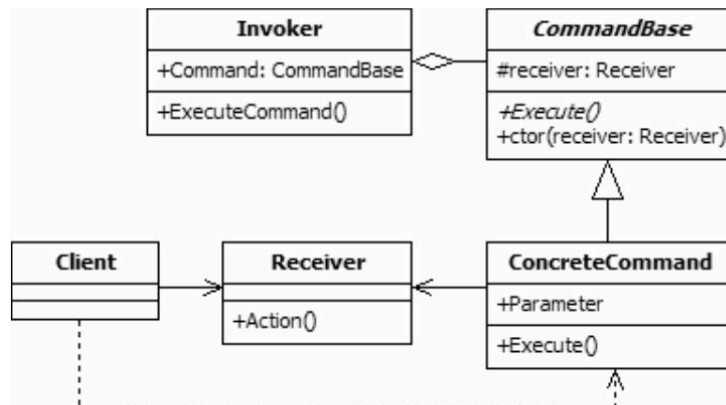
Applicazione in ambito determinato:

Esempio rilevante: Un telecomando universale che gestisce luci e TV di diversi produttori senza conoscerne le classi specifiche.

b) Class Diagram

1. **Command:** Interfaccia con il metodo `execute()`.
2. **ConcreteCommand:** Implementa `execute()` delegando l'azione al `Receiver`.
3. **Invoker:** Chiede al comando di eseguire la richiesta (es. il telecomando).
4. **Receiver:** L'oggetto finale che esegue l'azione (es. la lampada).
5. **Client:** Crea il `ConcreteCommand` e lo assegna al `Invoker`.

Command Pattern Class Diagram



c) Principi di programmazione soddisfatti: Soddisfa il principio del Disaccoppiamento (Decoupling): L'Invoker non conosce le classi dei dispositivi; conosce solo l'interfaccia Command. Questo permette di aggiungere nuovi dispositivi o funzioni senza modificare il codice del telecomando, seguendo anche l'Open-Closed Principle.

Adapter

a) Motivazione: Integrare una nuova libreria o classe di un fornitore (vendor) la cui interfaccia è diversa da quella attualmente attesa dal sistema. **Uso:** Convertire l'interfaccia di una classe in un'altra interfaccia che i client si aspettano, permettendo a classi incompatibili di lavorare insieme. **Funzionamento:** L'adapter agisce come intermediario; riceve le chiamate dal client tramite l'interfaccia conosciuta e le traduce in richieste comprensibili per la classe originale (adaptee).

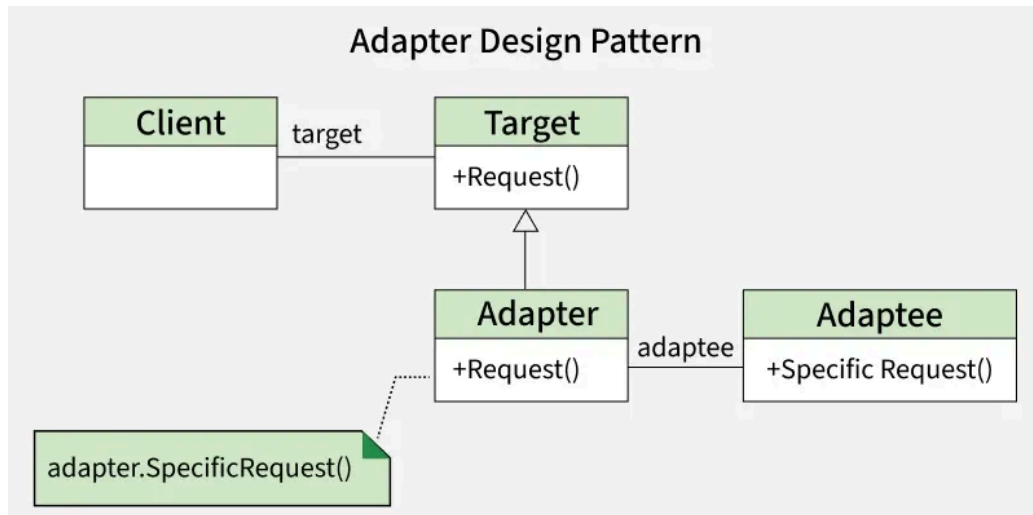
PseudoCodice:

Esempio rilevante: Adattare un display europeo (EUDisplay), che misura la temperatura in Celsius, a una stazione meteo americana (USAWeatherStation) che fornisce dati in Fahrenheit.

Applicazione in ambito determinato: Sistemi legacy, integrazione di librerie esterne o standard internazionali differenti.

b) Class Diagram:

Il diagramma prevede un Client collegato a un'interfaccia Target. L'Adapter implementa il Target e mantiene un'istanza dell'Adaptee (composizione). Le richieste del client al Target vengono delegate dall'Adapter all'Adaptee tramite la chiamata specifica attesa da quest'ultimo.



c) Principi di programmazione soddisfatti: Soddisfa il principio di incapsulamento del cambiamento di interfaccia: l'adapter avvolge l'oggetto originale per modificarne l'interfaccia senza che il client debba conoscere i dettagli della traduzione.

Facade

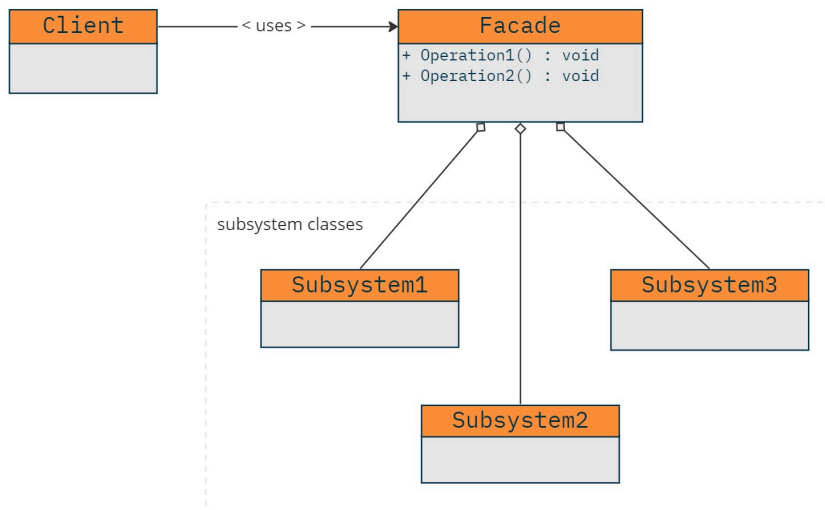
a) Motivazione: Semplificare l'uso di un sottosistema complesso che richiede l'interazione coordinata di molte classi diverse. **Uso:** Fornire un'interfaccia unificata e di alto livello a un insieme di interfacce in un sottosistema, rendendolo più facile da usare. **Funzionamento:** La Facade "avvolge" un set di oggetti di un sottosistema e offre metodi semplici che delegano il lavoro alle classi sottostanti.

PseudoCodice:

Esempio rilevante: Un sistema di Home Theater. Per "guardare un film", la Facade coordina l'accensione di amplificatore, proiettore, lettore DVD, abbassa lo schermo e regola le luci

Applicazione in ambito determinato: Sistemi tecnologici per casa

b) Class Diagram: Mostra il Client che comunica esclusivamente con la classe Facade. La Facade è composta da vari riferimenti alle classi dell'Existing Sub-System (es. Amplifier, Projector) e ne coordina le chiamate.



c) Principi di programmazione soddisfatti: Soddisfa il Principio di Minima Conoscenza (Legge di Demeter): un oggetto dovrebbe parlare solo con i suoi "amici immediati". La Facade disaccoppia il client dal sottosistema complesso, riducendo le dipendenze.

Template

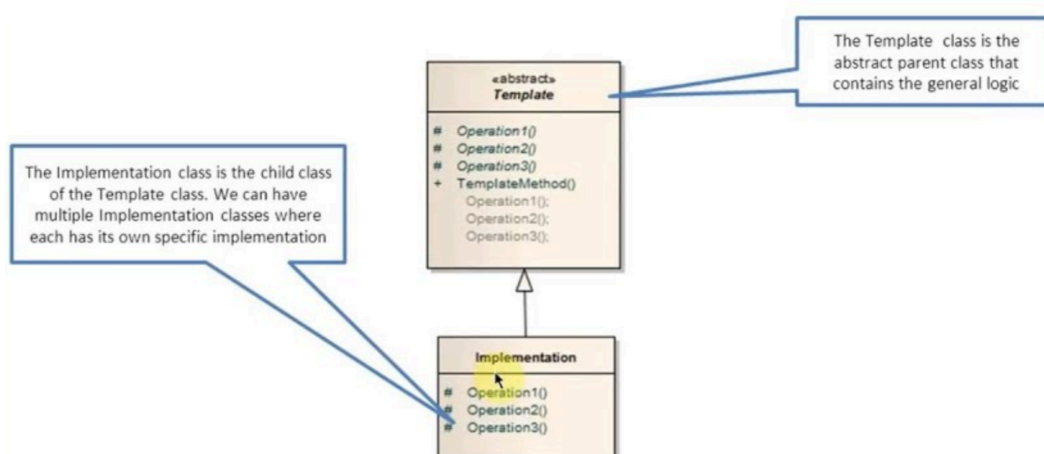
a) Motivazione: Evitare la duplicazione di codice quando più classi seguono lo stesso algoritmo di base, ma differiscono in alcuni dettagli implementativi. **Uso:** Definire lo scheletro di un algoritmo in un metodo della superclasse, lasciando che le sottoclassi implementino i passaggi specifici. **Funzionamento:** L'algoritmo principale è in un metodo final (per non essere modificato). I passaggi variabili sono dichiarati come metodi astratti o "hooks" (metodi con implementazione predefinita vuota) che le sottoclassi possono sovrascrivere.

PseudoCodice:

Esempio rilevante: Preparazione di tè e caffè. Entrambi bollono acqua e versano in tazza (comuni), ma differiscono nell'infusione e nell'aggiunta di condimenti.

Applicazione in ambito determinato: algoritmi di ordinamento (tipo array.sort)

b) Class Diagram: Una AbstractClass definisce il TemplateMethod() e le PrimitiveOperation() astratte. Le ConcreteClass implementano queste operazioni senza poter cambiare la struttura dell'algoritmo dettata dal template.



c)Principi di programmazione soddisfatti: Soddisfa il Principio di Hollywood: "Non chiamateci, vi chiameremo noi". La superclasse (alto livello) controlla l'algoritmo e chiama le sottoclassi (basso livello) solo quando necessario per i dettagli.

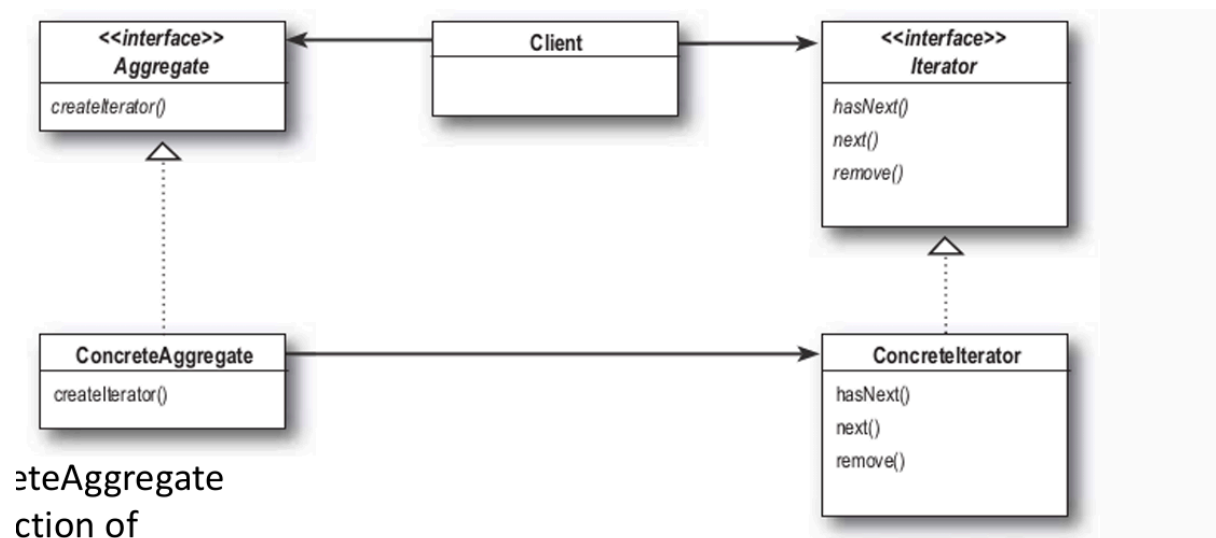
Iterator

a)Motivazione: Accedere agli elementi di un oggetto aggregato in modo sequenziale senza esporre la sua struttura interna (es. se usa un array o una lista).**Uso:** Quando si hanno diverse collezioni e si desidera un modo uniforme per scorrerle, permettendo al codice client di essere polimorfico rispetto ai tipi di aggregati.**Funzionamento:** Si incapsula l'iterazione in un oggetto separato ("Iterator") che implementa un'interfaccia standard con metodi come hasNext() e next().

PseudoCodice:

Esempio rilevante: Unire i menù di due ristoranti dove uno memorizza i piatti in un ArrayList e l'altro in un semplice Array.

b)Class Diagram: L'aggregato definisce un metodo createIterator(). L'interfaccia Iterator definisce le operazioni di scorrimento, implementate da classi concrete per ogni tipo di collezione.



c)Principi di programmazione soddisfatti: Soddisfa il principio della Singola Responsabilità: scorpora il compito della traversata dall'aggregato, lasciando a quest'ultimo solo il compito di gestire la collezione di elementi.

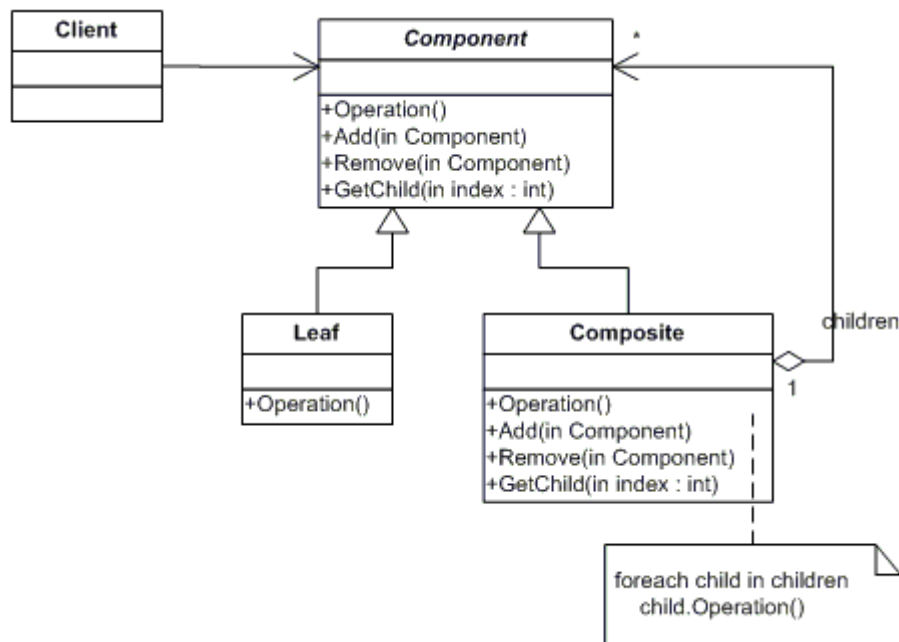
Composite

a) Motivazione: Consentire ai client di trattare singoli oggetti e composizioni di oggetti in modo uniforme.**Uso:** Quando è necessario rappresentare gerarchie di oggetti a struttura ad albero (parte-tutto).**Funzionamento:** Sia gli elementi singoli ("Leaf") che i contenitori ("Composite") implementano la stessa interfaccia Component.

PseudoCodice:

Esempio rilevante: Un menu che può contenere sia singoli piatti che interi sotto-menu (es. menu dessert).

b)Class Diagram:Prevede un'interfaccia Component comune. La classe Leaf rappresenta gli elementi finali, mentre Composite contiene una collezione di figli e delega loro le operazioni.



c)Principi di programmazione soddisfatti:Soddisfa il principio di Trasparenza: il client non deve preoccuparsi se sta interagendo con una foglia o con un nodo composto, semplificando la logica del codice esterno.

State

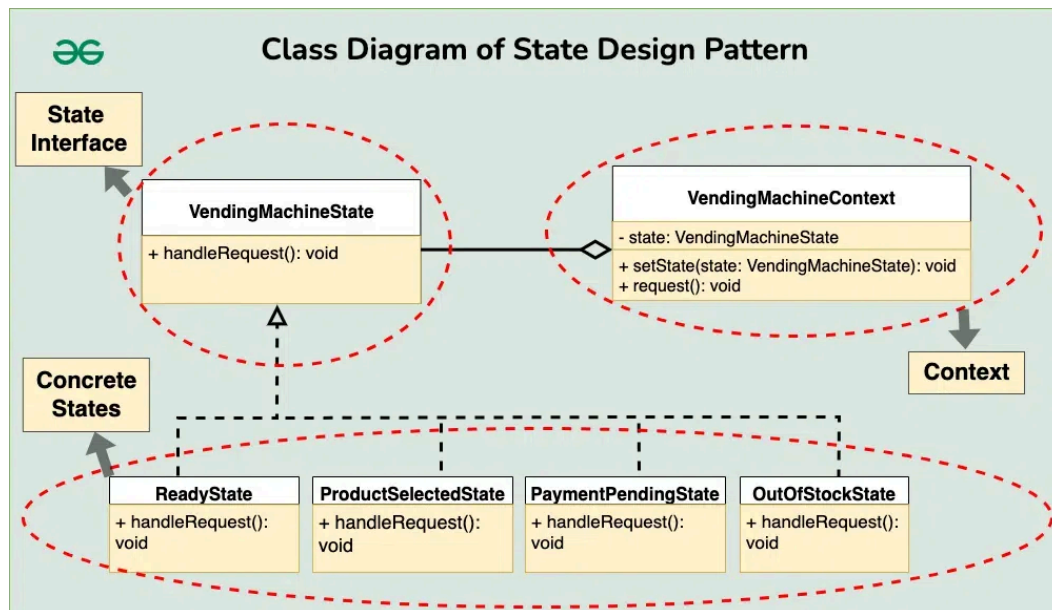
a)Motivazione: Permettere a un oggetto di cambiare il suo comportamento quando il suo stato interno cambia, facendo apparire l'oggetto come se avesse cambiato classe.**Uso:** Per evitare strutture condizionali massive (if-else, switch) che dipendono dallo stato dell'oggetto.**Funzionamento:** Si incapsula ogni stato in una classe separata. L'oggetto principale ("Context") delega il comportamento all'oggetto stato corrente.

PseudoCodice:

Esempio rilevante: Una macchina di gomme da masticare che si comporta diversamente (eroga, rifiuta moneta, ecc.) a seconda che sia "Senza moneta", "Con moneta" o "Vuota".

Applicazione in ambito determinato: Macchine a stati finiti, workflow di approvazione documenti, gestione di connessioni di rete.

b)Class Diagram:La classe Context mantiene un riferimento a un'interfaccia State. Diverse classi ConcreteState implementano i metodi dell'interfaccia definendo i comportamenti per quello stato specifico.



c)Principi di programmazione soddisfatti:Soddisfa il principio Open-Closed: è possibile aggiungere nuovi stati senza modificare il codice delle classi esistenti o della classe Context.

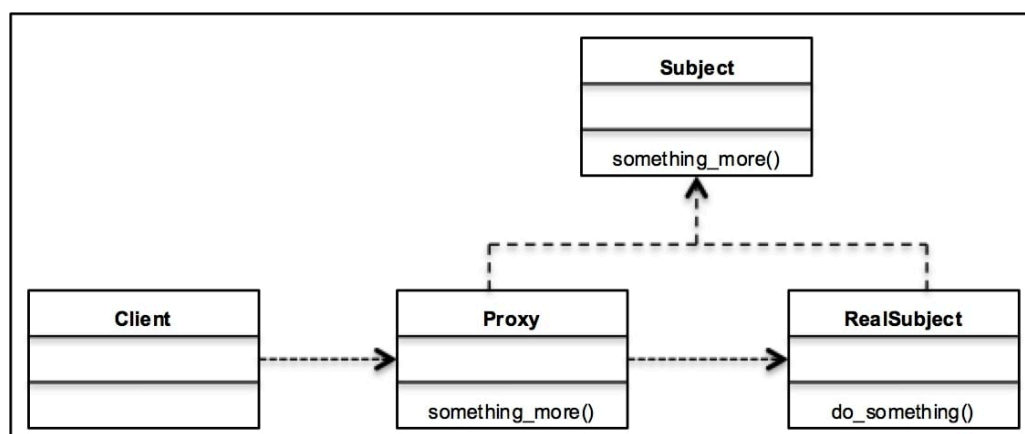
Proxy

Motivazione: Fornire un rappresentante o un sostituto di un altro oggetto per controllarne l'accesso. **Uso:** Per gestire oggetti remoti, oggetti costosi da creare (lazy loading) o per proteggere l'accesso a risorse sensibili. **Funzionamento:** Il Proxy implementa la stessa interfaccia del "RealSubject". Riceve le richieste del client, esegue operazioni accessorie (es. autenticazione o caricamento) e poi le inoltra all'oggetto reale.

PseudoCodice:

Esempio rilevante: Caricamento di immagini da remoto; il Proxy mostra un'icona di caricamento finché l'immagine reale non è stata completamente scaricata.

b)Class Diagram: Client, Subject (interfaccia comune), RealSubject e Proxy. Il Proxy mantiene un riferimento al RealSubject.



c)Principi di programmazione soddisfatti:Soddisfa il principio di Controllo dell'Accesso e Trasparenza: il client non sa di parlare con un proxy anziché con l'oggetto reale, ma il proxy può gestire in autonomia complessità come la rete o la sicurezza.

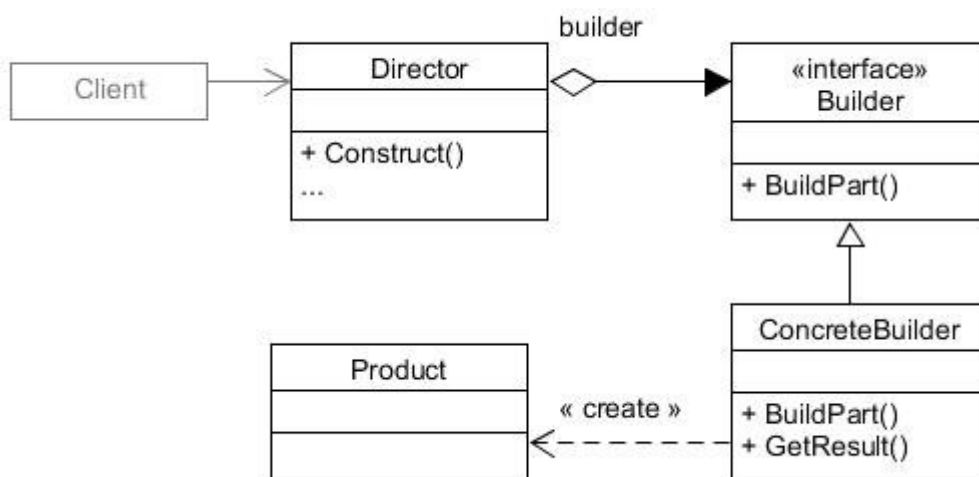
Builder

a)Motivazione: Incapsulare la costruzione di un prodotto complesso, permettendo che il processo di costruzione possa creare diverse rappresentazioni.**Uso:** Quando la creazione di un oggetto richiede molti passaggi coordinati o quando si vuole nascondere la complessità interna del prodotto al client.**Funzionamento:** Un oggetto Director coordina i passaggi della costruzione chiamando un'interfaccia Builder generica; le sottoclassi concrete del builder realizzano i singoli pezzi del prodotto.

PseudoCodice:

Esempio rilevante: La creazione di una mappa per un videogioco, dove un MazeBuilder aggiunge in sequenza muri, stanze, tesori e mostri.

b) Class Diagram: Comprende il Director (che gestisce l'ordine della costruzione), l'interfaccia Builder (con i metodi per creare le parti), i ConcreteBuilder (che implementano i metodi e mantengono il prodotto finale) e il Product stesso.



c)Principi di programmazione soddisfatti: Incapsulamento della creazione: Tutta la logica di assemblaggio è nascosta all'interno del Builder, isolando il client dai dettagli costruttivi. Separazione delle responsabilità: Il Director sa "quando" costruire, mentre il Builder sa "come" costruire ogni singolo pezzo.

Bridge

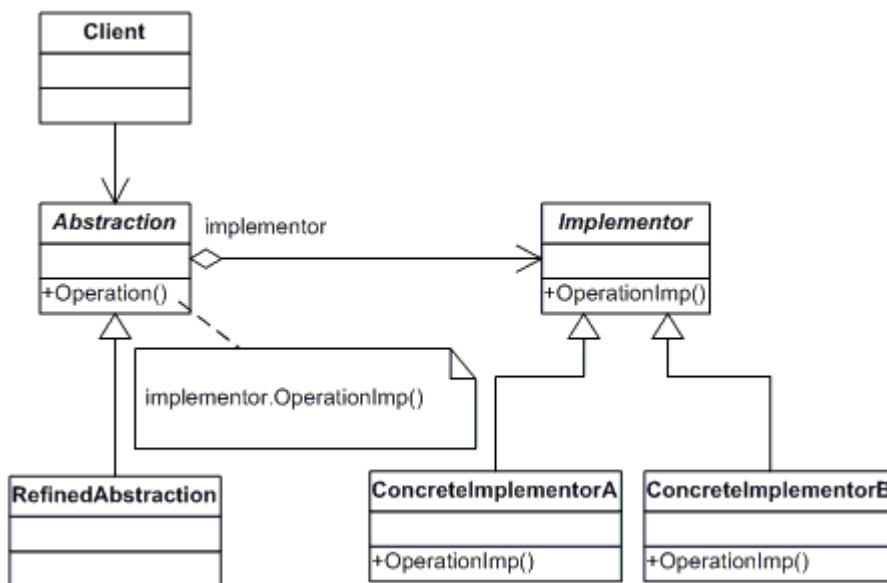
a)Motivazione: Viene utilizzato per variare non solo le implementazioni ma anche le astrazioni in modo indipendente.**Uso:** Evita che un'astrazione sia legata permanentemente a un'implementazione, permettendo a entrambe di cambiare senza influenzarsi a vicenda.

Funzionamento: Separa l'astrazione dall'implementazione ponendole in gerarchie di classi distinte.

Esempio rilevante: La progettazione di un telecomando ergonomico e intuitivo (Astrazione) che deve interfacciarsi con molti modelli diversi di TV (Implementazioni).

Applicazione in ambito determinato: Sistemi dove l'interfaccia utente (astrazione) e la logica di controllo del dispositivo (implementazione) evolvono a ritmi diversi.

b)Class Diagram:Il diagramma mostra una classe Abstraction che mantiene un riferimento a un'interfaccia Implementor. Le RefinedAbstraction estendono l'interfaccia utente, mentre i ConcreteImplementor forniscono il codice specifico per i diversi dispositivi



c)Principi di programmazione soddisfatti:Favorire la composizione rispetto all'ereditarietà: Invece di creare una sottoclasse per ogni combinazione di telecomando/TV, il telecomando ha un riferimento alla TV. Disaccoppiamento: L'astrazione e l'implementazione sono isolate, permettendo modifiche a una senza dover ricompilare l'altra.

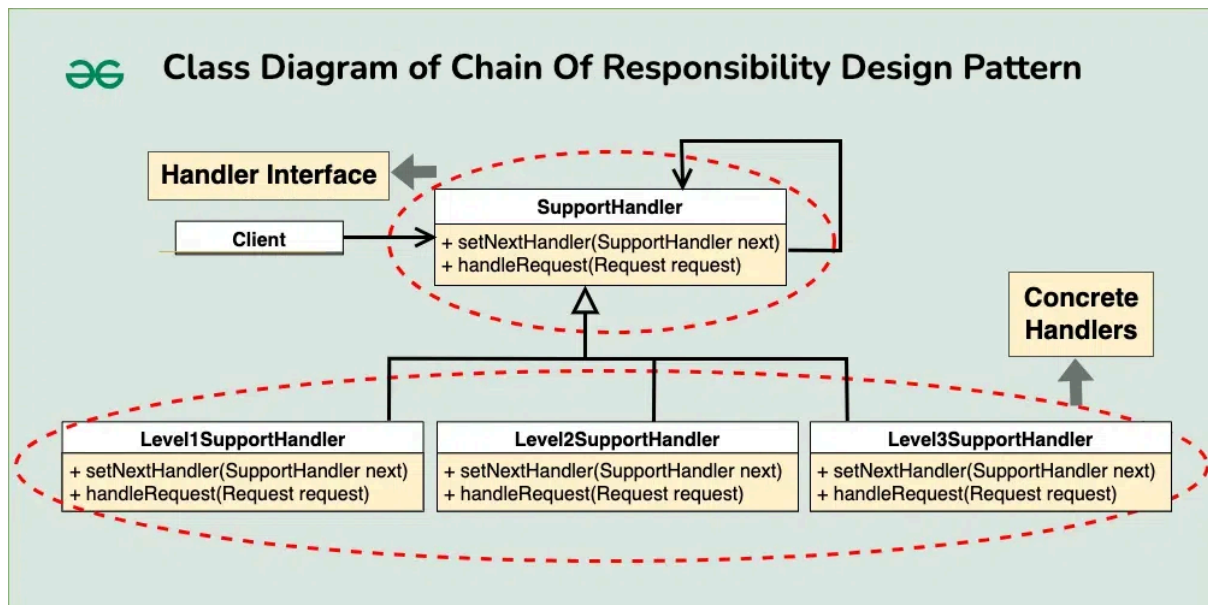
Chain of Responsibility

a)Motivazione: Disaccoppiare il mittente di una richiesta dal destinatario, dando a più oggetti la possibilità di gestire la richiesta.**Uso:** Quando una richiesta deve essere elaborata da uno tra diversi gestori possibili, ma non si sa in anticipo quale sarà quello corretto.**Funzionamento:** Gli oggetti gestori sono concatenati; ogni oggetto riceve la richiesta e decide se elaborarla o passarla al successore nella catena.

PseudoCodice:

Esempio rilevante: Un sistema di gestione delle email che riceve un messaggio e lo passa a una catena di filtri: il filtro "Spam" lo analizza, se non è spam lo passa al filtro "Fan Mail", poi al filtro "Complaint" e infine all'ufficio acquisti.

b) Class Diagram: Prevede una classe astratta Handler che definisce l'interfaccia per gestire le richieste e mantiene un riferimento (successor) a un altro Handler. I ConcreteHandler implementano la logica specifica di gestione.



c) Principi di programmazione soddisfatti: Basso accoppiamento (Loose Coupling): Il mittente non conosce l'identità del gestore finale, sa solo che la richiesta verrà processata lungo la catena. Single Responsibility Principle: Ogni gestore nella catena è responsabile solo di un tipo specifico di operazione o filtro.

4) Indicare le principali differenze / somiglianze tra due pattern di design

Factory:

a) Differenza tra simple factory e static factory

Somiglianze:

- Astrazione della creazione: nascondono la logica di istanziazione (new) al client
- Disaccoppiamento: Il client non ha bisogno di conoscere le classi concrete specifiche, ma solo l'interfaccia o la classe astratta comune
- Punto unico di controllo: Se la logica di creazione cambia, devi modificare solo la factory

Differenze:

- Uno è una classe a parte, l'altro è un metodo statico
- La classe può essere estesa, il metodo no
- Generalmente il metodo della static factory è all'interno della classe stessa (statico) ed è più descrittivo (es: Datetime.now al posto di new Datetime)

Composite

b) [NON Richiesto] implementazione Composite Iterator

✓ Design: Architecture and Methodology -> completa

1. Che cosa si intende per design di alto livello di dettaglio

Per design di alto livello intendiamo le scelte architetturelle fatte a monte in base ai requisiti (che siano funzionali o no). Vengono decisi i componenti principali del software, le sue proprietà generali e la gestione delle dipendenze al suo interno ed esterno

A livello di dettaglio andiamo a decidere in maniera modulare come strutturare ogni parte del software: c'è una grossa dipendenza dalle scelte architetturelle fatte in precedenza, e tutti i requisiti funzionali devono essere soddisfatti. Rientrano in questa categoria anche l'utilizzo di design pattern per scopi specifici nel codice

2. Quali sono i principi di programmazione SOLID? Fornire esempi dei principi di programmazione.

S - Single Responsibility (Responsabilità Singola): Una classe deve avere un solo motivo per cambiare. Deve svolgere un'unica funzione specifica.

O - Open/Closed (Aperto/Chiuso): Il codice deve essere aperto alle estensioni ma chiuso alle modifiche. Aggiungi nuove funzionalità senza cambiare il codice già testato.

L - Liskov Substitution (Sostituzione di Liskov): Le sottoclassi devono poter sostituire le classi base senza alterare la correttezza del programma.

I - Interface Segregation (Segregazione delle Interfacce): Meglio tante interfacce piccole e specifiche piuttosto che una singola interfaccia enorme. Un client non deve essere forzato a dipendere da metodi che non usa.

D - Dependency Inversion (Inversione delle Dipendenze): Bisogna dipendere dalle astrazioni, non dalle implementazioni concrete. I moduli di alto livello non devono dipendere da quelli di basso livello.

esempi: <https://www.baeldung.com/solid-principles>

3. Principi per il design di componenti: principi relativi alla coesione e all'accoppiamento. Misurare la qualità del design dei componenti.

Principi di design di componenti:

- Reuse/Release: i componenti devono avere un numero di release, devono essere compatibili tra loro (in una versione) e gli sviluppatori devono essere al corrente dei cambiamenti. Implicazioni: i componenti devono essere "raggruppabili" in una maniera che sia sensata per sviluppatori e utenti.
- Common Closure: applicazione del principio S ma ai componenti. Un componente deve cambiare per un solo motivo. Implicazioni: classi che cambiano assieme vanno nello stesso componente.
- Common Reuse: raggruppare solo le classi legate tra loro. Si cerca di evitare singoli elementi (classi) per un solo componente.

Implicazioni: si cerca di stare attenti alle dipendenze di ogni classe in modo da non avere troppi componenti.

I Primi due sono "inclusive" perché tendono a rendere i componenti più grandi. Il terzo è "exclusive" perché li rende più piccoli.

L'obiettivo è quello di trovare l'equilibrio che si trova tra questi 3.

Misurare la qualità è possibile tramite metriche di stabilità: numero di dipendenze in entrata e uscita (fan-in e fan-out) ($\text{instabilità} = \text{Fout} / \text{Fin} + \text{Fout}$)

Un'altra metrica è il livello di astrazione: Na / Nc (Nc =numero di classi nel componente, Na =numero di classi astratte nel componente)

Si cerca sempre di stare in zone utili: molto astratto e instabile oppure concreto e molto stabile.

4. Che cosa è l'architettura del software? In che modo può essere caratterizzata?

Non c'è una definizione condivisa: in generale, l'architettura software è tutto ciò che è importante. (altre def: organizzazione fondamentale del sistema ad alto livello, decisioni di design che vanno prese all'inizio del progetto)

- Caratteristiche architetturali
cosa deve essere supportato (scalabilità, testabilità, manutenibilità, supporto)
Le caratteristiche dell'architettura software sono le qualità fondamentali che determinano come sono soddisfatti i requisiti non funzionali e in generale i compiti del sistema.
- Decisioni architetturali
Scelte chiave che influenzano la struttura del sistema (tipo di db, metodo di comunicazione tra servizi / componenti...)
Sono utilizzate come vincoli per guidare lo sviluppo. Sono molto rigide e vincolanti, si tende a farne il meno possibile. Sono essenziali al fine del successo.
- Componenti logici
Rappresentano i blocchi funzionali che compongono il software e le loro interazioni. Rappresentate da directory o namespace, contengono codice sorgente che implementa quella specifica funzionalità
- Stile architetturale
Struttura generale e forma che prende il sistema. può essere vista come un piano che dà forma al prodotto. ES: event-driven, microservices, monolith...

5. Cosa si intende per caratteristiche architetturali (e quali sono le principali), decisioni architetturali, componenti logici e stili?

Risposto nella 4)

6. Qual'è la differenza tra architettura e design?

L'architettura si occupa della struttura del sistema: user-interface, comunicazione tra servizi, database...

Il design si concentra sul "come": viene descritto tramite UML e non influisce sulla struttura di base. Non si occupa della struttura fisica del codice.

Le decisioni strategiche, che richiedono molto impegno per essere seguite e con molti trade-off tendono a essere architetturali.

Decisioni più semplici e con meno trade-off sono inserite nel design.

7. Quali sono le principali caratteristiche architetturali?

Caratteristiche architetturali:

- Modularità: quanto il software è composto da componenti discreti
- Testabilità: completezza nel testing (unit testing, funzionale, UAT ed esplorativo)
- Distribuibilità: facilità ed efficienza nella distribuzione.
- Estensibilità: quanto è facile estendere il codice per gli sviluppatori.
- Abilità di disaccoppiamento: divisione delle parti che devono essere autonome per avere dei benefici

8. Stili architetturali: layered, microkernel, event driven, microservizi,... principali caratteristiche, pro e contro di ogni architettura

layered

Abbiamo strati isolati che comunicano solo con quello inferiore. Sono presenti eccezioni in casi di utilities (come logging) o stati condivisi

Pro:

- semplicità di sviluppo (ci si può facilmente dividere in team per ogni strato)
- singola responsabilità di ogni stato
- veloce (essendo monolita non abbiamo chiamate di rete)

Contro:

- Scalabilità ridotta o nulla
- Testabilità difficile a causa di una grande unica codebase
- Rischio di creare molte dipendenze: spesso i monoliti diventano reti intricate di accoppiamenti non necessari

Microkernel

Abbiamo un core che è il sistema principale, generalmente stabile, piccolo e raramente affetto da cambiamenti. I plugin invece implementano funzionalità aggiuntive e sono rilasciati in maniera indipendente.

Il core dà a disposizione un'interfaccia ai plugin, in modo che essi non siano legati obbligatoriamente all'implementazione del core stesso

Pro:

- Estendibilità e adattabilità molto buone: basta inserire un nuovo plugin
- Struttura semplice: core e plugin
- Divisione dei compiti: la struttura semplice permette di avere una chiara suddivisione delle responsabilità tra plugin e core

Contro:

- Il core deve essere molto stabile e cambiare poco (romperebbe i plugin)
- Condivisione tra plugin: possono crearsi reti intricate di plugin che dipendono l'uno dall'altro rendendo tutto il sistema complesso e pesante
- Performance (come detto prima)

Microservices

Servizi distribuiti come unità in maniera indipendente, ognuno con una specifica funzione. Comunicano generalmente usando REST.

Bisogna trovare un equilibrio nella dimensione del servizio: troppo piccolo causerà sicuramente dipendenze con altri, troppo grande perderebbe il vantaggio di tutta l'architettura.

Per decidere la granularità con cui distribuire i microservizi, valutiamo:

- coesione
- volatilità
- tolleranza ai guasti
- scalabilità
- sicurezza
- dipendenze dati e DB
- workflow e coordinazione

I primi 5 per aumentare la granularità, gli ultimi 2 per diminuirla (unire servizi assieme)

Si cercano di usare delle shared library per codice comune, stando attenti ai cambiamenti perché impattano tutti i servizi dipendenti da essa.

Si usano due modi per gestire le comunicazioni:

Orchestrator:

- gestione centralizzata delle comunicazioni e dello stato
- problema di scalabilità e performance: tutti dipendono dall'orchestrator
- mancata alta disponibilità: con un problema del servizio centrale tutti sono offline
- facilitazione a cambiamenti del workflow e alla gestione errori

Choreography:

- Controllo distribuito tra tutti i servizi: non c'è un master centrale
- scalabilità, performance e problemi di dipendenze sono migliorati: tutto è isolato e indipendente quindi più facilmente allargabile.
- difficile controllo dello stato del software, del controllo errori o di un eventuale recupero di un microservizio offline

Pro e contro:

- Scalabilità indipendente dei singoli componenti.
- Resilienza ai guasti parziali del sistema.
- Flessibilità tecnologica per ogni servizio.
- Complessità operativa e di monitoraggio elevata.
- Latenza dovuta alle chiamate di rete tra servizi.
- Difficoltà nella gestione della consistenza dei dati distribuiti.

Event driven

Divisione del software in servizi che rispondono ad un certo evento.

La comunicazione avviene tramite un canale apposito in maniera asincrona.

La comunicazione degli eventi non richiede conoscenza di chi è in ascolto (o comunque di chi è il destinatario), generalmente sono inviati con dei topic tramite un sistema publisher-subscriber. Un evento lanciato da un utente è detto iniziatore, mentre gli altri derivati.

Si può usare un DB centrale condiviso (lentezza per un piccolo cambiamento e problema di eventuale memoria condivisa), un DB per domain (crea dipendenze tra un servizio e un altro nel caso siano necessari scambi di dati) oppure uno per servizio (peggiori performance a causa della gestione del DB per ogni servizio e alla latenza nella comunicazione).

Due tipi di topologia di comunicazione: con mediator (fa da orchestratore delle comunicazioni tramite una coda, e si occupa di mandare i messaggi ai servizi iscritti) oppure con broker (non c'è un mediatore centrale, adatta a sistemi che richiedono basse latenze e performance. si ha poco controllo e in caso di errore è difficile risalire alla sorgente)

Pro e contro:

- Elevata reattività e disaccoppiamento tra i componenti.
- Scalabilità orizzontale semplificata e flessibilità nell'aggiungere nuove funzionalità.
- Gestione efficace di carichi di lavoro asincroni in tempo reale.
- Difficoltà nel monitoraggio e nel debug dei flussi.
- Rischio di complessità nella gestione della consistenza eventuale dei dati.

9. Principi clean dell'architettura del software: Indipendenza, disaccoppiamento dei layer / disaccoppiamento degli use case

Disaccoppiamento per layer:

applico il Single Resp. Principle e Common Closure per dividere cose che cambiano per motivi diversi. es: business logic e UI devono cambiare in maniera completamente separata.

I dettagli tecnici (come DB, query language...) NON devono essere collegati alla business logic

Disaccoppiamento per case:

Diversi use case dividono naturalmente il software in diverse porzioni. Es: aggiungere ed eliminare un utente

La divisione per use-case deve essere immaginata come linee verticali che dividono i layer orizzontali del sistema

Indipendenza:

Tenendo alto il livello di indipendenza, abbiamo molti benefici:

- disponibilità e resistenza agli errori
- scalabilità
- maggiore performance e bandwidth (se sfruttata correttamente)

Questo si può raggiungere facendo scelte a livello di architettura (SOA, Service oriented Architecture)

In generale si cerca di ridurre la duplicazione (vera) usando i due metodi per layer e per use-case. Nel caso di servizi simili ma che si sviluppino in modo diverso è utile e necessario tenerli separati, anche a costo di quella che può sembrare dipendenza (falsa)

Livelli di disaccoppiamento:

1. Sorgente: evitare una completa ricompilazione del progetto alla modifica di una dipendenza
2. Rilascio: ogni servizio deve essere un'unità indipendente nel rilascio. Non devo aver bisogno di riavviare tutto al rilascio della singola unità.
3. Servizio: la comunicazione deve avvenire solo via rete e ogni servizio deve essere completamente indipendente in esecuzione e in codice.

10. Principi clean dell'architettura del software: policy e livelli / entità

la Policy è la logica che governa il sistema. Il software non è un blocco unico, ma un insieme di policy divise per Livelli di Astrazione:

Livelli Alti: Sono le parti che contengono le business rules (il "perché" del software). Sono le più distanti dagli input/output.

Livelli Bassi: Sono i dettagli implementativi (database, UI, framework, dispositivi). Sono le parti che cambiano più spesso.

Regola di Dipendenza: Il codice di livello superiore non deve mai conoscere il codice di livello inferiore. Ad esempio, la logica di calcolo di un mutuo non deve sapere se i dati arrivano da un database SQL o da un file CSV.

I componenti formano un DAG (directed acyclic graph) in cui i nodi sono le policy allo stesso livello e i bordi sono le dipendenze tra componenti a livelli diversi.

Flussi di dati: mostrati come frecce continue curve.

Dipendenze del codice sorgente: mostrate come linee tratteggiate dritte.

Entità: oggetti che contengono regole di business critiche insieme ai dati associati. Il principio è quello di tenere assieme regole e dati essenziali, rendendoli indipendenti dall'implementazione vera e propria.

Use cases: regole specifiche di un'applicazione, legate al funzionamento (più di basso livello). Specificano come e quando l'input dell'utente viene processato e dato in output. In generale, descrivono le interazioni tra Entità.

11. Principi clean dell'architettura del software: principali componenti dell'architettura

L'architettura clean si basa su cerchi concentrici che isolano la logica di business dai dettagli tecnici. Al centro si trovano le entità che contengono le regole di business fondamentali e indipendenti. Attorno ad esse operano i casi d'uso che orchestrano il flusso dei dati e le logiche specifiche dell'applicazione. Seguono gli adapter che convertono i dati dai formati interni a quelli esterni per database e web. Infine ci sono i driver e i framework che gestiscono gli strumenti tecnologici.

L'ultimo componente è il Main: il suo unico compito è creare le factory o gli strategy e poi lasciare il controllo alle parti di alto livello del sistema.

Le dipendenze puntano esclusivamente verso l'interno garantendo che il nucleo rimanga stabile e testabile.

Ci sarebbe una parte di documentazione ma non è richiesta nella domanda.

✓ Design Characteristics and Metrics → COMPLETA

1) In che modo può essere descritta e caratterizzata la progettazione di un software?

Non esiste una definizione univoca di buon design. Un buon design è caratterizzato da diversi attributi.

Dai requisiti:

- Coerenza in tutto il design:

Interfaccia utente comune (aspetto e flusso logico); Elaborazione degli errori comune; Report comuni; Interfacce di sistema comuni; Guida comune; Tutto il design è portato allo stesso livello di profondità.

- Completezza del design:

Tutti i requisiti sono presi in considerazione; Tutte le parti del design sono portate a termine allo stesso livello di profondità.

L'ingegneria del software iniziale si concentra sulla progettazione dettagliata e sugli attributi a livello di codifica, non sulla progettazione architettonica come fa ora che si concentra su tutti gli artefatti. Programmi e moduli sono stati a lungo considerati gli artefatti più importanti, ora no, ora è tutto l'insieme. Per non avere più una descrizione soggettiva sono state introdotte le metriche, anche se un minimo di giudizio personale rimane. Misurazioni e metriche erano mirate agli elementi di progettazione dettagliata e di codifica.

Le caratteristiche di un buon design potrebbero essere: Facile da capire; Facile da modificare; Facile da riutilizzare; Facile da testare; Facile da integrare; Facile da codificare. Secondo Yourdon e Constantine (1979), queste nozioni possono essere misurate da due caratteristiche: coesione e accoppiamento.

2) Cosa si intende per metrica del software, quali sono i limiti nell'utilizzare metriche per caratterizzare i software e quali tipi di metriche possono essere utilizzate

"Lo scopo delle metriche software è quello di effettuare valutazioni durante l'intero ciclo di vita del software per confermare se i requisiti di qualità del software vengono soddisfatti." L'utilizzo di metriche software riduce la soggettività nella valutazione e nel controllo della qualità del software, fornendo una base quantitativa per prendere decisioni sulla qualità del software. Tuttavia, l'utilizzo di metriche software non elimina la necessità del giudizio umano nelle valutazioni del software. Si prevede che l'utilizzo di metriche software all'interno di un'organizzazione o di un progetto abbia un effetto benefico rendendo la qualità del software più visibile."

Le metriche vengono applicate utilizzando tecniche di misurazione, producendo valori misurati. Le metriche però hanno anche dei limiti ovvero è difficile definire "intervalli normali" generali per i valori misurati delle singole metriche di qualità del software. Gli intervalli normali possono essere determinati empiricamente per uno specifico progetto software. → non abbiamo dei range ottimali.

Le metriche possono fare più male che bene. I valori misurati suggeriscono precisione, ma c'è una forte tendenza da parte dei professionisti a mostrare eccessivo ottimismo e troppa sicurezza. → a volte possono essere controproducenti, sovrastimando l'importanza delle misure. Gli sviluppatori possono adattare il loro lavoro alle metriche se utilizzate come rigorosi strumenti di qualità, con conseguente comportamento opportunistico e guidato dalle

metriche. → le metriche sono misure deterministiche e dipendono dal linguaggio di programmazione che stai usando

Esempio: le linee di codice (LOC) misurano la dimensione di un'unità di codice sorgente contando le righe.

Una metrica di qualità del software è una metrica software che caratterizza gli aspetti qualitativi. L'interpretazione dei valori misurati è fortemente empirica

- Metriche di prodotto → Caratterizzano un prodotto intermedio o finale del processo di sviluppo software.

Esempi: dimensione del codice, complessità del codice, soddisfazione del cliente.

- Metriche di processo → Caratterizzano metodi, tecniche e strumenti utilizzati nello sviluppo, implementazione e manutenzione del sistema software.

Esempi: efficacia della rimozione dei difetti durante lo sviluppo, tempo di risposta dei processi di correzione dei bug.

- Metriche di progetto → Caratterizzano il progetto in termini di tempo, budget e gestione.

Esempi: numero di sviluppatori software, schema di assegnazione del personale durante il ciclo di vita del software, costi, tempi, produttività.

Nota: una specifica metrica software può appartenere a più categorie.

3) Cosa si intende per analisi statica e dinamica del software

Analisi statica del codice sorgente → Si riferiscono e riflettono aspetti della qualità e della complessità della codifica e della modularizzazione. Ovvero valutano il codice senza la sua esecuzione (più facile).

Esempi: Linee di codice (LOC, NLOC); Livello di annidamento (NL); Complessità ciclomatica (McCabe); Complessità di Halstead; Accoppiamento; Coesione; Duplicazione del codice; Punti funzione.

Analisi dinamica → basata sulla registrazione delle dinamiche di sistema in fase di esecuzione. Si riferiscono e riflettono aspetti del comportamento del sistema. Ovvero valutano il codice con la sua esecuzione.

Esempi: Copertura del codice; Bug per riga di codice; Utilizzo delle funzionalità.

4) Caratteristiche delle misure basate

a) Sul conteggio delle linee di codice

Le metriche delle linee di codice (LOC) sono un insieme di metriche software che misurano quantitativamente la dimensione del codice sorgente contando le "linee di codice sorgente".

Le metriche LOC possono essere applicate a tutti i tipi di unità di codice come funzioni, metodi, classi, pacchetti o file. Calcolato tramite analisi statica del codice sorgente.

Le metriche LOC vengono utilizzate per determinare l'"ordine di grandezza"

dell'implementazione di un sistema informatico complesso come misura statistica per stimare e prevedere gli sforzi. Per costruire metriche di livello superiore, ad esempio per stimare i costi e gli sforzi generali (ad esempio, per manutenzione, gestione degli errori, correzione dei bug). Le metriche LOC possono essere utilizzate per determinare quantitativamente il grado di modifica o evoluzione del codice sorgente nel tempo.

Esistono diverse varianti delle metriche LOC in base alla definizione di "riga di codice sorgente":

LOC: linee fisiche: una riga corrisponde a una riga nella rappresentazione testuale del codice sorgente

SLOC: righe di codice sorgente, senza commenti e righe vuote

CLOC: righe di codice con commenti

Svantaggi: Il LOC può essere correlato allo sforzo ma non alla funzionalità, ovvero non fornisce informazioni sulla reale complessità di un'implementazione. Ad esempio, lo sviluppo di funzioni può richiedere giorni, ma comporta meno di 1.000 LOC.

Il LOC non considera la complessità derivante dalla generazione automatica di codice utilizzando la metaprogrammazione o i framework.

Se il LOC viene utilizzato come misura dell'attività, gli sviluppatori possono facilmente compromettere tale misura con "stili di codifica opportunistici", come la scrittura di codice prolisso.

LOC contiene anche codice duplicato che è una cosa negativa.

È difficile confrontare il LOC di unità di codice sorgente scritte in diversi linguaggi di programmazione.

b) Sul livello di innestamento del codice

La metrica del livello di annidamento (NL) è una metrica software che misura quantitativamente la complessità del codice sorgente misurando la profondità di inclusione delle strutture di controllo l'una nell'altra. I codici sorgente applicano l'annidamento in diverse forme. Il livello di annidamento indica il livello di contenimento (profondità) di un blocco logico di codice sorgente. Utilizzato per misurare la complessità di unità di codice sorgente come funzioni o metodi.

NL: Livello di annidamento più alto di una data unità di codice sorgente

NL#+: Numero di righe di codice al di sopra di un certo livello di annidamento, a partire da # (ad esempio, NL4+, NL5+) → queste linee di codice innestate non le scelgo io ma sono date da per esempio se uso una funzione di una libreria che usa altre funzioni...

Valori elevati di NL indicano generalmente che è necessario uno sforzo maggiore per comprendere l'unità di codice rispetto alla comprensione e alla comprensione del programma. L'osservazione di fondo è che il codice profondamente annidato diventa estremamente difficile da comprendere da un punto di vista cognitivo ma anche da correggere e da mantenere. Le misurazioni metriche di NL possono essere utilizzate per identificare quelle parti del codice sorgente che dovrebbero essere riprogettate utilizzando strategie di modularizzazione, astrazione o incapsulamento.

c) Sulla complessità del codice (Halstead e complessità ciclomatica di McCabe)

Sviluppato da Halstead negli anni '70 per analizzare la complessità del codice sorgente dei programmi. Si basano sull'analisi statica del codice sorgente che si concentra sulla complessità lessicografica del codice sorgente.

Operatori e operandi sono definiti dalla loro relazione reciproca. Il presupposto di base di Halstead è che la programmazione richieda sforzi cognitivi (passaggi cognitivi elementari) che possono essere quantitativamente correlati a operatori e operandi.

La metrica di Halstead utilizza quattro unità metriche per l'analisi di un programma sorgente.

$n1$ = numero di operatori distinti - $n2$ = numero di operandi distinti - $N1$ = somma di tutte le occorrenze di $n1$ - $N2$ = somma di tutte le occorrenze di $n2$

Da queste unità fondamentali vengono sviluppate tantissime altre misure come la difficoltà:

$\text{Difficoltà} = (n1/2) * (N2/n2)$

Le metriche di Halstead misurano in realtà solo la complessità lessicale del programma sorgente e non la struttura o la logica.

La misura della complessità ciclomatica per il software è il risultato dell'osservazione di T. J. McCabe secondo cui la qualità del programma è direttamente correlata alla complessità del flusso di controllo o al numero di rami nel codice sorgente.

È rappresentata come segue:

Complessità ciclomatica = $E - N + 2p$, dove E = numero di archi del grafo, N = numero di nodi del grafo (gruppi indivisibili di comandi di un programma) e p = numero di componenti connessi (solitamente $p = 1$).

Il numero di complessità ciclomatica di McCabe può anche essere calcolato:

Complessità ciclomatica = numero di decisioni binarie + 1

Complessità ciclomatica = numero di regioni chiuse + 1

Definizione: Misura la complessità della progettazione strutturale all'interno di un programma.

Ci indica anche la pianificazione dei test per valutare il numero di casi di test necessari per coprire ogni punto decisionale. Lasciamo indietro le parti del test che si verificano di più, andiamo a controllare le parti più nascoste.

5) Quali sono i problemi generati dalla duplicazione del codice

Violazione del principio DRY. La clonazione del codice aumenta lo sforzo di manutenzione. Le modifiche devono essere apportate in modo coerente più volte se il codice è ridondante. Spesso non è documentato dove è stato copiato il codice. La ricerca manuale del codice copiato non è fattibile per sistemi di grandi dimensioni.

Durante l'analisi, lo stesso codice deve essere letto più e più volte, quindi confrontato con l'altro codice solo per scoprire che questo codice è già stato analizzato. Se il codice fosse stato implementato una sola volta in una funzione, questo sforzo avrebbe potuto essere completamente evitato.

Esistono diversi tipi di duplicazione:

Tipo 1: Frammenti di codice identici, ad eccezione delle variazioni in spazi, layout e commenti.

Tipo 2: Frammenti sintatticamente identici, ad eccezione delle variazioni in identificatori, letterali, tipi, spazi, layout e commenti.

Tipo 3: Frammenti copiati con ulteriori modifiche come istruzioni modificate, aggiunte o rimosse, oltre alle variazioni in identificatori, letterali, tipi, spazi, layout e commenti.

Tipo 4: Due o più frammenti di codice che eseguono lo stesso calcolo, ma sono implementati da varianti sintattiche diverse.

Sono presenti plugin nell'IDE per rilevare le duplicazioni del codice.

6) Come si possono misurare le relazioni tra i moduli del software

Le relazioni tra i moduli del software si possono misurare con le dipendenze. Una dipendenza denota una relazione formale tra due moduli, possono essere rilevate tramite l'analisi statica del codice sorgente. Il modulo A dipende dal modulo B, ad esempio, se A chiama B (dipendenza di chiamata)

A eredita da B (dipendenza di ereditarietà)

A utilizza una variabile di tipo B (dipendenza dati/tipo di dati)...

Le caratteristiche di un buon design potrebbero essere:

Facile da capire; Facile da modificare; Facile da riutilizzare; Facile da testare; Facile da integrare; Facile da codificare.

Secondo Yourdon e Constantine (1979), queste nozioni possono essere misurate da due caratteristiche: coesione e accoppiamento.

Dipendenza interna ed esterna

L'accoppiamento di un modulo si riferisce alle dipendenze esterne da altri moduli, ovvero al grado di dipendenza inter-modulo.

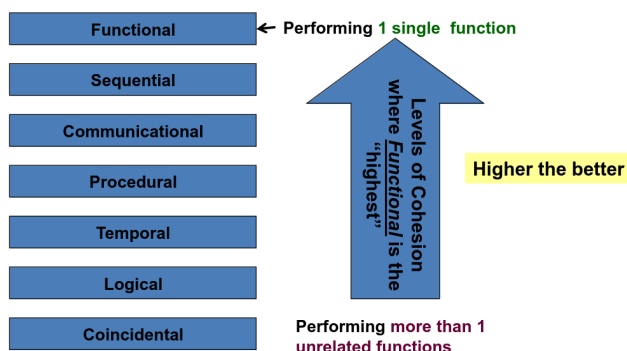
La coesione di un modulo si riferisce alle dipendenze interne all'interno dello stesso modulo, ovvero al grado di dipendenza intra-modulo.

Questi due concetti non sono lontani dalle misure di complessità menzionate in precedenza.

La coesione affronta le caratteristiche intra-modulo in modo simile alle metriche di Halstead e McCabe. L'accoppiamento affronta le caratteristiche inter-modulo in modo simile alle misure di flusso informativo fan-in e fan-out di Henry-Kafura e alla misura di complessità di Card e Glass.

In generale, ci impegniamo per una forte coesione e un accoppiamento debole nel nostro progetto.

7) Cosa si intende e come si descrive la coesione e l'accoppiamento dei moduli di un software



Per mantenere la semplicità del design, dobbiamo assicurarci che ogni unità del progetto sia focalizzata su un singolo scopo.

Coesione: Attributo di un'unità di progettazione di alto livello o di dettaglio che identifica il grado di appartenenza o relazione degli elementi all'interno di tale unità.

I componenti di un'unità di progettazione altamente coesa sono tutti correlati nel perseguire un singolo scopo o una singola funzione.

Coesione casuale: l'unità di progettazione o di codice esegue più attività non correlate.

Coesione logica: il progetto esegue una serie di attività simili, la relazione tra gli elementi esiste, ma è relativamente debole.

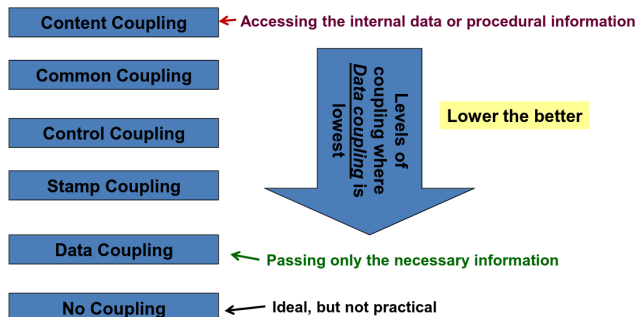
Coesione temporale: raggruppa elementi correlati nel tempo (routine di inizializzazione).

Coesione procedurale: gli elementi sono correlati proceduralmente, collegati da una sequenza di controllo.

Coesione comunicazionale: simile alla coesione procedurale, ma le attività sono mirate agli stessi dati o set di dati. Ha una maggiore vicinanza interna rispetto alla coesione procedurale.

Coesione sequenziale: l'unità di progettazione svolge un'attività principale, ma la delimitazione di una "singola" attività non è sempre chiara.

Coesione funzionale: l'unità di progettazione raggiunge un obiettivo chiaro. Essendo orientata a un singolo obiettivo, richiede pochissime modifiche per il riutilizzo se è necessario lo stesso obiettivo. Raggiungere una perfetta coesione funzionale non è sempre fattibile. → metrica coesione: Program Slice- and Data Slice-Based



Accoppiamento: Grado di interazione e interdipendenza tra unità software. Anche con unità altamente coese, un certo grado di accoppiamento è inevitabile. Le classi strettamente accoppiate sono difficili da: Riutilizzare in isolamento; Comprendere in modo indipendente; Modificare o correggere senza conoscere tutti i moduli dipendenti. Un'elevata interdipendenza aumenta il rischio che un errore in un modulo influenzi gli altri. Conclusione: Un elevato accoppiamento non è auspicabile nella progettazione del software.

Accoppiamento di contenuto: due unità accedono reciprocamente ai dati o alle procedure interne → qualsiasi modifica in un'unità richiede un'attenta revisione dell'altra

Accoppiamento comune: due unità fanno riferimento alla stessa variabile globale.

Accoppiamento di controllo: un'unità passa informazioni di controllo per influenzare il comportamento di un'altra unità.

Accoppiamento di timbro: un'unità passa un gruppo di dati a un'altra unità → Possono essere scambiati più dati del necessario.

Accoppiamento di dati: solo i dati necessari vengono passati tra le unità.

Bassa interdipendenza, considerato debolmente accoppiato.

Metrica dell'accoppiamento: Fenton and Melton Measure

La coesione e l'accoppiamento vanno in conflitto

8) Quali sono le principali misure che caratterizzano un software costruito con un linguaggio Object Oriented (CK metrics)

Una classe è semplicemente un'entità software che vogliamo mantenere semplice attraverso una forte coesione. L'interrelazione e l'interazione tra le classi dovrebbero essere mantenute semplici attraverso un accoppiamento lasco. Nell'OO, esistono, tuttavia, alcune estensioni al modo in cui misureremo questi attributi.

Le metriche CK misurano la complessità della progettazione in relazione all'impatto sugli attributi di qualità esterna come manutenibilità e riutilizzabilità, validati teoricamente ed empiricamente.

WMC (Metodi pesati per classe): somma pesata di tutti i metodi in una classe.

DIT (Profondità dell'albero ereditario): lunghezza massima dell'ereditarietà da una classe alla sua radice → l'ereditarietà profonda può rendere le classi difficili da trovare e comprendere.

NOC (Numero di figli): numero di sottoclassi immediate di una classe.

CBO (Coupling Between Objects): numero di classi a cui una classe è accoppiata.

RFC (Response For a Class): insieme di metodi coinvolti nella risposta a un messaggio inviato a un oggetto della classe.

LCOM (Lack of Cohesion of Methods): misura la correlazione tra metodi e variabili di istanza locali di una classe.

(Mancanza di coesione LCOM:

0 → quando un metodo accede a tutti gli attributi, è quello che vogliamo

1 → quando un metodo accede ad un solo attributo, non è quello che vogliamo

Le interfacce sono importanti perché sono quelle che hanno più a che fare con l'utente, anche esse vengono valutate tramite test)

✓ Implementation → COMPLETA

1) Che cosa si intende per fase di implementazione e quali sono le caratteristiche di questa fase? In che modo le caratteristiche si influenzano e quando assumono una particolare importanza? Che importanza è opportuno dare all'ottimizzazione del codice?

Per implementazione si intende la trasformazione della progettazione (generalmente sviluppata tramite grafici UML in maniera esaustiva) in un programma scritto in un particolare linguaggio di programmazione.

Generalmente, alcuni dettagli molto specifici sono lasciati come parte dell'implementazione e non durante la progettazione.

Caratteristiche di una buona implementazione:

Leggibilità

Manutenibilità

Prestazioni

Tracciabilità: tutti gli elementi nel codice devono corrispondere a parti del design iniziale

Correttezza: il codice fa quello per cui è stato ideato

Completezza: sono rispettati i requisiti

La performance deve arrivare per ultima: un'ottimizzazione prematura può causare gravi danni in termini di leggibilità e manutenibilità

Quando si ottimizza bisogna avere benchmark per analisi accurate, in modo da decidere in maniera corretta quali porzioni migliorare. Inoltre, è sempre necessaria un'analisi costi-benefici: il guadagno che avrò è maggiore del costo dell'ottimizzazione?

2) Nozione dello standard ISO/IEC 25010

Definisce due modelli:

- *Modello di qualità del software*: è composto da 8 caratteristiche misurabili internamente o esternamente (quindi durante lo sviluppo e a prodotto completo)
 - Stabilità funzionale
 - Efficienza prestazionale
 - Compatibilità
 - Usabilità
 - Affidabilità
 - Sicurezza
 - Manutenibilità
 - Portabilità
- *Modello di qualità in uso*: 5 caratteristiche misurate in un contesto reale, in modo che non dipendano solo dal software ma anche da utenti, piattaforme e ambiente di utilizzo
 - Efficacia: accuratezza e completezza con il quale l'utente raggiunge gli obiettivi prefissati
 - Efficienza: rapporto tra risorse usate e efficacia del software
 - Soddisfazione: quanto è utile / comodo / importante per il nostro utente il nostro prodotto? quanta fiducia ripone in esso?
 - Assenza di rischio: Come il nostro software mitiga rischi ambientali, economici e a persone
 - Copertura del contesto: il grado di efficienza, efficacia, soddisfazione e assenza di rischio nel nostro contesto. Inoltre, quanto è flessibile ad eventuali cambiamenti non previsti nei requisiti

3) Cosa si intende per stile di programmazione?

Lo stile di programmazione sono delle linee guida definite prima dell'implementazione. Fanno parte dello stile convenzioni che riguardano:

- nomi di variabili
 - indentazione
 - commenti
 - divieto di specifiche funzionalità del linguaggio (es: non usare l'ereditarietà multipla)
- Alcune buone pratiche: usare software per la formattazione automatica, essere consistenti nello stile, spezzare grandi funzioni in piccole (meglio per testing e il principio Single Responsibility).

4) Come possono essere classificati i commenti?

1. ripetizioni di codice: evitare ad ogni costo (il codice potrebbe cambiare)
2. spiegazione del codice: da evitare, il codice dovrebbe spiegarsi da solo se abbastanza chiaro
3. markers: segnalano TODO da implementare o cambiamenti. Meglio usare un sistema di versionamento
4. Riassunti di blocchi di codice: sono ok ma vanno tenuti aggiornati
5. descrizioni dell'intento del codice: buoni, voglio spiegare il "perchè" e non il "come"
6. referenze esterne: solitamente link a risorse per spiegare un certo passaggio

5) ●●●Cosa si intende per debugging e quali sono le fasi di quel processo?

Il debugging è il processo di individuazione e rimozione di errori nel codice

1. Riproduzione
Cerco di riprodurre in modo consistente l'errore. Può essere molto difficile in contesti molto grandi e complessi (esecuzione su macchine o reti diverse)
2. Localizzazione
Trovo la specifica porzione di codice che causa il problema, generalmente tramite log e l'uso di debuggers. E' la fase che richiede il maggior tempo, solitamente.
3. Correzione
Modifica del codice per la rimozione del problema. Importante capire al 100% la causa prima della modifica per non introdurre nuovi bug.
4. Verifica
Verificare che non si siano creati altri bug a causa della mia correzione. Si ri-eseguono i vecchi test case per vedere se il codice funziona ancora come previsto prima della modifica (regression testing)

6) Che cosa si intende per programmare per asserzioni? In che modo Java supporta quel processo? Quali sono le funzionalità offerte da junit per il debugging del software?

Programmare per asserzioni significa dichiarare in modo esplicito le precondizioni, cioè le condizioni necessarie per permettere una corretta esecuzione del codice.

Java supporta le asserzioni:

- `assert exp` → se `exp` è false solleva un `AssertionError` senza informazioni sull'errore
- `assert exp : exp2` → se `exp` è false solleva un `AssertionError` passando la rappresentazione in stringa di `exp2` come messaggio informativo

esempi su dove usarle:

- prima o dopo dei loop (voglio essere sicuro sul valore di qualche variabile) (loop-invariant)
- dove il codice (teoricamente) non arriva mai: se si raggiunge quel punto, c'è un bug (control-flow invariant)

7) Cosa si intende per defensive programming?

Il defensive programming è la pratica per cui si scrive codice anticipando errori futuri. L'idea è quella di pensare che i bug ci saranno e di predirli o renderli identificabili in maniera semplice.

Alcuni esempi di defensive programming:

- String literal first: in una comparazione tra stringhe, si evita una possibile NullPointerException
- Se una funzione ritorna -1 se c'è un errore e numeri positivi altrimenti, usare ≥ 0 piuttosto che $== -1$ (potrebbero essere inseriti altri errori)
- Prima di controllare la lunghezza di un elemento, controllare se è null (stesso problema del string literal first)
- usare asserzioni
-

8) Cosa si intende per refactoring del codice?

Per refactoring descriviamo il cambiamento della struttura del codice senza cambiare nulla del suo comportamento esterno. E' effettuato per migliorare manutenibilità e leggibilità.

Si fa refactoring quando incontriamo:

- codice duplicato
- metodi/funzioni lunghe
- classi troppo grandi: non rispettano i principi SOLID
- switch: generalmente possono essere sostituiti usando il polimorfismo
- violazioni dell'incapsulazione: una classe usa metodi/attributi privati di un'altra
- una classe/metodo dipende troppo da un'altra esterna, dove non necessario

Possibili soluzioni:

- estrazione (metodi): divido un blocco in un metodo a se stante
- algoritmo di sostituzione: sostituisco un blocco con un nuovo algoritmo più chiaro, senza cambiare il funzionamento
- spostamento: sposto qualcosa in una classe dove ha più senso che risieda
- estrazione (classi): divido una classe in 2 o più (sempre per i principi SOLID)

9) Indicare le principali linee guida per lo sviluppo del codice

Credo di aver risposto nella 1

10) Indicare alcune tecniche di refactoring con possibili esempi (ad esempio estrazione di un metodo, estrazione di una superclasse, ...)

Ho risposto nella 8

✓ Testing and Quality Assurance

1) Cosa si intende per qualità del software e come si ottiene un software di qualità. In che modo/ attraverso quali prospettive si può valutare la qualità (cosa si intende per verifica e per validazione)?

La qualità del software è un concetto ampio che si riferisce a quanto un software soddisfa le aspettative degli utenti e rispetta determinati standard tecnici, prestazionali e di sicurezza quindi quanto è affidabile, manutenibile e conforme alle esigenze degli utenti. Un software è definito di qualità se rispetta queste 2 caratteristiche principali:

- Il Quality Assurance (QA) include tutte le attività per misurare e migliorare la qualità. Copre l'intero processo di sviluppo, dalla pianificazione e progettazione al testing e all'implementazione. Il QA include anche la formazione, la definizione dei processi e la preparazione del team di sviluppo.
Il suo obiettivo è prevenire i difetti prima che si verifichino. → **verifica**
- Il Quality Control (QC) si concentra sulla verifica della qualità del prodotto. Implica il rilevamento e la correzione dei difetti prima del rilascio.
Il QC garantisce che il software finale soddisfi gli standard di qualità definiti. → **validazione**

Verifica

È il processo di controllo che si occupa di garantire che il software **sia stato sviluppato correttamente** e che rispetti le specifiche tecniche, i requisiti funzionali e le normative. In altre parole, "stiamo costruendo il prodotto nel modo giusto?". La verifica riguarda solitamente il **"come"** il software è stato costruito, ed è un processo che si concentra sull'identificazione di errori nel codice, nei dati e nella progettazione.

Esempi di attività di verifica: Test del codice, ispezioni del design, revisione del codice, ecc.
Strumenti di verifica: Test automatici, controlli di qualità del codice, strumenti di analisi statica.

Validazione

È il processo che si assicura che il software soddisfi **le aspettative degli utenti** e i requisiti di business. In altre parole, "stiamo costruendo il prodotto giusto?". La validazione riguarda l'**accertamento** che il software soddisfi i bisogni reali dell'utente finale e le sue aspettative.
Esempi di attività di validazione: Test di accettazione degli utenti, simulazioni, feedback dai clienti.

Strumenti di validazione: Testing con utenti reali, test di usabilità, prove sul campo.

2) Come si possono individuare gli errori in un software o in un artefatto a esso connesso

1. Test → Il processo di progettazione di casi di test, esecuzione del programma in un ambiente controllato e verifica della correttezza dell'output. Si concentra sul comportamento dinamico e rileva errori durante l'esecuzione.
2. Ispezioni e revisioni → Applicate sia al codice che agli artefatti software intermedi. Comportano la revisione paritaria, richiedendo la partecipazione di più persone oltre all'autore originale. Mirano a individuare difetti in anticipo mediante l'esame umano di documenti o codice.

3. Metodi formali → Utilizzano tecniche matematiche per dimostrare la correttezza di un programma. Forniscono solide garanzie, ma sono raramente utilizzate in contesti commerciali o aziendali a causa della loro complessità e dei costi.
4. Analisi statica → Analizza la struttura statica del codice o dei documenti senza eseguirla. Solitamente automatizzata, identifica errori o modelli soggetti a errori nelle prime fasi dello sviluppo.

3) Differenza tra fallimento, difetto e errore

Errore (Error)

È l'azione sbagliata commessa da una persona (sviluppatore, analista, progettista).
Può essere un errore di comprensione, di progettazione, di scrittura del codice. → Origine del problema.

Esempio:

Il programmatore interpreta male un requisito o scrive male una formula.

Difetto (Fault / Bug)

È la manifestazione dell'errore nel software, cioè il problema inserito nel codice o nel progetto.

Il difetto è un errore "materializzato" nel prodotto. → Conseguenza dell'errore.

Esempio:

A causa dell'errore del programmatore, nel codice viene scritto:

```
if (x = 5) invece di if (x == 5).
```

Fallimento (Failure)

È il comportamento errato del software durante l'esecuzione causato da uno o più difetti.

Un difetto può rimanere nascosto per molto tempo, ma quando si manifesta durante l'esecuzione diventa un fallimento. → Risultato visibile all'utente.

Esempio:

L'app si blocca o restituisce un risultato sbagliato quando l'utente inserisce certi valori.

4) Cosa si intende per testing del software? Quali sono gli scopi del testing? In che modo è possibile fare del test? Esistono delle procedure automatizzate per supportare il testing?

Il testing può essere definito come: un processo di analisi di un software per rilevare le differenze tra le condizioni esistenti e quelle richieste (ovvero difetti/errori/bug) e per valutare le caratteristiche del software (standard ANSI/IEEE 1059 - inattivo).

Il testing è un'attività svolta per valutare la qualità del prodotto e per migliorarla, identificando difetti e problemi. (Guida al Software Engineering Body of Knowledge - 2004).

Il test è l'esame sistematico e metodico di un prodotto di lavoro utilizzando una varietà di tecniche, con l'obiettivo specifico di dimostrare che il prodotto soddisfa lo scopo previsto. Viene eseguito in un ambiente che riproduce da vicino le operazioni reali, garantendo che il comportamento osservato rifletta ciò che gli utenti sperimenteranno in produzione.

Caratteristiche principali

- Sistematico: il test deve essere pianificato e organizzato, non casuale.
- Metodico: segue un processo definito con fasi e obiettivi chiari.

- Prodotto di lavoro: il test si applica non solo al codice, ma anche a requisiti, documenti di progettazione e specifiche.
- Varietà di tecniche: include revisioni, analisi statica, test dinamici, simulazioni, ecc.

Scopi del testing:

1. Verificare che tutti i requisiti siano stati soddisfatti.
2. Verificare che l'elemento sottoposto a test sia completo e si comporti come previsto dagli utenti e dagli stakeholder.
3. Infondere fiducia nella qualità dell'elemento sottoposto a test.
4. Individuare tempestivamente guasti e difetti, per evitare che raggiungano la produzione.
5. Fornire informazioni a supporto del processo decisionale, ad esempio se il software è pronto per il rilascio.
6. Ridurre il rischio associato a una qualità del software inadeguata.
7. Garantire la conformità ai requisiti contrattuali, legali o standard

Limiti del testing

1. Il testing non può dimostrare che un prodotto funzioni sempre. Dimostra solo che il prodotto funziona nei casi specifici testati. Un testing corretto fornisce una ragionevole certezza che il software funzioni in situazioni simili, ma non in tutti i casi possibili.
2. I risultati del testing possono verificare se gli obiettivi di qualità sono stati raggiunti.

Il test si può effettuare attraverso vari approcci (black, white, grey-box), diversi livelli (unit, integration, system, acceptance) e tipologie (funzionali, non funzionali, regressione...).

Oggi il testing è ampiamente supportato da strumenti automatizzati che facilitano il controllo continuo della qualità anche in ambienti di sviluppo complessi.

5) Chi esegue il testing? Quando si fa il testing? Cosa si intende per testing funzionale e non funzionale?

Chi esegue il test?

- I programmatori progettano ed eseguono casi di test per verificare il proprio codice. Questa attività è in genere definita unit testing. L'obiettivo è garantire che i singoli componenti funzionino correttamente prima dell'integrazione.
- I tester sono professionisti tecnici che progettano, implementano ed eseguono casi di test. Alcuni tester analizzano anche statisticamente i risultati dei test per valutare la qualità del prodotto. Spesso supportano le decisioni di rilascio fornendo dati sui livelli di rischio residuo e di difetto.
- Gli utenti partecipano ai test per identificare problemi di usabilità ed esporre il software a scenari di utilizzo reali. Gli utenti possono anche partecipare ai test di accettazione:
Alpha testing: eseguito da utenti all'interno dell'organizzazione di sviluppo.
Beta testing: eseguito da utenti esterni all'organizzazione.

Quando iniziare i test?

Iniziare i test il prima possibile nel processo di sviluppo del software. Iniziare tempestivamente riduce i costi e le lavorazioni e aiuta a fornire al cliente un software privo di

errori. I test dovrebbero iniziare nella fase di raccolta dei requisiti e continuare fino alla distribuzione. I test assumono forme diverse in ogni fase del ciclo di vita dello sviluppo del software (SDLC):

Fase dei requisiti: analisi e verifica dei requisiti.

Fase di progettazione: revisione e miglioramento della progettazione del software.

Fase di implementazione: gli sviluppatori testano il codice completato (test unitari).

Il test è quindi un'attività continua, non un passaggio finale.

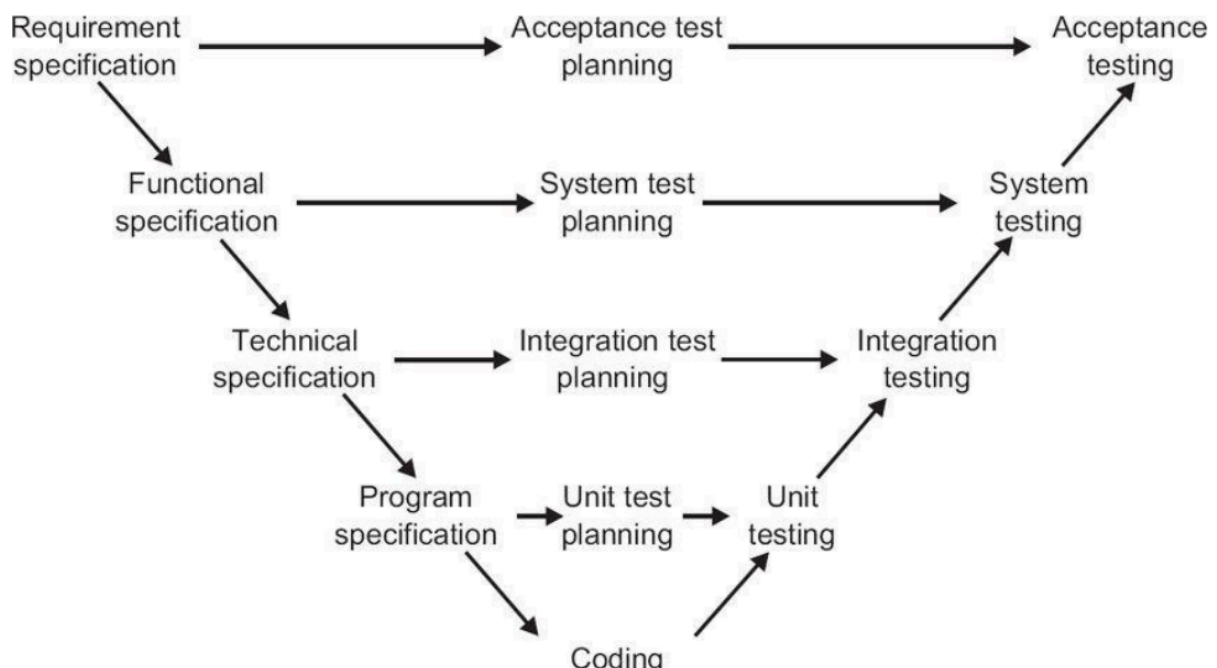
Test statico → si riferisce al test senza eseguire il codice.

Test dinamici → Comporta l'esecuzione del programma con dati di test per verificarne il comportamento.

Test funzionale (analisi requisiti funzionali) → Esamina la funzionalità specifica di un sistema, verifica che cosa il software fa. Include controlli di completezza, correttezza e appropriatezza. Il test funzionale viene eseguito a tutti i livelli di test, dal test unitario fino al test di accettazione. Il test funzionale è chiamato test basato sulle specifiche o test black-box. Può essere misurato in termini di percentuale di requisiti coperti dai test. La progettazione di test a questo livello richiede spesso competenze di dominio specifiche.

Test non funzionali (analisi requisiti non funzionali) → Verifica come il software si comporta. Si concentra sugli attributi comportamentali e qualitativi del sistema, come usabilità, prestazioni, efficienza e sicurezza. Dovrebbe essere eseguito a tutti i livelli di test per rilevare potenziali difetti il prima possibile. Utilizza spesso tecniche di test black-box. In genere richiede conoscenze e competenze specialistiche. I risultati possono essere misurati.

6) Che differenza c'è tra testing a livello di requisito (acceptance test), a livello funzionale (System test), a livello di specifica (Integration test), a livello di codice (Unit test)?



Esistono fasi diverse dello sviluppo del codice e spesso sono correlate ad uno specifico test.

UNIT TESTING: all'interno di questo livello vengono valutate le singole classi in modo indipendente per valutare se siano soddisfatti le specifiche di programma al fine di ridurre i rischi dello sviluppo. L'obiettivo è rilevare problemi nel flusso dei dati o costrutti di codice eccessivamente complessi, tramite un approccio Test-Driven Development (TDD), definendo prima i test poi procedendo con lo sviluppo effettivo del codice.

INTEGRATION TESTING: le classi indipendenti vengono raggruppate per formare altri componenti o il sistema complessivo, per cui vengono distinti ulteriormente i due omonimi livelli. Si focalizzano sulle interfacce e le interazioni tra componenti dello stesso sottosistema o su micro servizi e pacchetti di sistemi interconnessi. Hanno il fine di verificare le specifiche tecniche, rilevando problemi di sequenziamento delle chiamate alle interfacce DB o API. È solitamente attuato dagli stessi sviluppatori per verificare l'integrazione tra componenti.

SYSTEM TESTING: in questo livello di testing viene verificata la bontà dell'intero sistema integrato verificandone il comportamento a livello di specifiche funzionali, quindi cercando di rilevare flussi di lavoro non previsti tramite una maggiore visione di insieme. Sono utilizzati per compiere decisioni sul rilascio e replicano in modo realistico l'ambiente di produzione.

ACCEPTANCE TESTING: si tratta di verificare che il sistema abbia raggiunto tutti i requisiti richiesti dall'utente, pertanto richiede la collaborazione di quest'ultimo. Infatti ne esistono varie forme tra le quali ricordiamo lo User (UAT) eseguito dall'utente, l'Operational (OAT) che valuta se il prodotto è pronto per il rilascio, il Contract and Regulatory (CRAT) che ne verifica gli aspetti contrattuali e legali, infine Alpha e Beta Testing rispettivamente svolto dentro o fuori dall'azienda produttrice. Risulta essenziale per valutare i fallimenti non funzionali, come ad esempio vulnerabilità, problemi di performance o incompatibilità.

7) Come si scelgono i casi da testare?

La scelta dei casi di test è una delle attività più critiche nel processo di testing, perché da essa dipende la capacità di individuare difetti in modo efficace e con un uso efficiente del tempo e delle risorse. I casi di test si selezionano combinando diverse tecniche, ciascuna con i propri obiettivi e vantaggi.

- Test basati sull'intuizione

Si basano sull'esperienza e sull'intuizione piuttosto che su metodi formali. Comune nei test Alpha e Beta, dove gli utenti o il personale di supporto testano il prodotto liberamente.

- Test basati sulle specifiche (test Black-Box)

Si basano interamente sulle specifiche senza esaminare il codice. Si concentrano sul comportamento di input/output del sistema.

Le tecniche comuni includono: Partizionamento delle classi di equivalenza (ECP) e Analisi dei valori limite (BVA).

- Test basati sul codice (test White-Box)

Utilizzano la conoscenza del codice sorgente per progettare i test. Mirano a raggiungere obiettivi di copertura (ad esempio, copertura di istruzioni, rami o percorsi).

- Casi di test esistenti (test di regressione)

Riutilizza casi di test creati in precedenza per versioni di sistema precedenti. Verifica che le nuove modifiche non interrompano le funzionalità esistenti.

- Tecniche basate sugli errori

Individuazione degli errori: progettazione di test per attivare probabili errori in base all'esperienza storica.

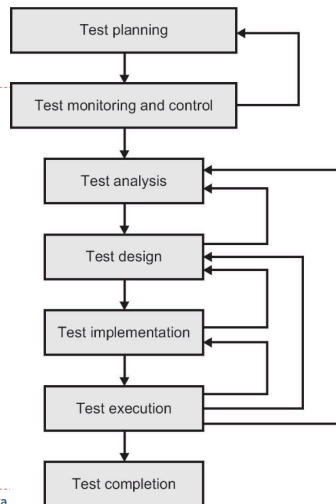
Analisi soggetta a errori: identificazione delle aree soggette a difetti tramite revisioni e ispezioni,

quindi concentrando gli sforzi di test su tali aree

Debug → corregge i problemi. Test → garantisce che le correzioni e il sistema nel suo complesso siano corretti.

Il test precoce fa risparmiare tempo e denaro. Nel test precoce, cerchiamo di individuare errori e difetti prima che vengano trasferiti alla fase successiva del processo di sviluppo.

8) Come avviene il processo di testing?



Il processo di testing si divide in 7 passaggi:

1. Pianificazione dei test

La pianificazione definisce cosa verrà testato e come verrà raggiunto.

Comprende: stabilire criteri di completamento (uscita) dei test, definire obiettivi dei test, selezionare l'approccio per raggiungere gli obiettivi, considerando limiti contestuali, creare un piano di test con scadenze chiare.

L'obiettivo è garantire che i test siano organizzati, tracciabili e allineati con gli obiettivi del progetto.

2. Monitoraggio e controllo dei test

Il monitoraggio implica il monitoraggio dei progressi, verificando se i risultati ottenuti corrispondono alle attività pianificate.

Comprende: verificare se i risultati ottenuti corrispondono ai test pianificati, valutare se sono stati raggiunti i criteri di uscita, decidere se servono test aggiuntivi.

L'obiettivo è assicurarsi che il processo di testing sia conforme agli obiettivi.

3. Analisi dei test

Identificare cosa testare, basandosi sui documenti di riferimento.

Comprende: esaminare la base di test (requisiti, progettazione, rischi), identificare difetti come ambiguità o contraddizioni, dare priorità alle condizioni di test da verificare, stabilire tracciabilità bidirezionale tra base di test e condizioni di test.

L'obiettivo è assicurarsi che tutte le caratteristiche siano coperte e tracciabili.

4. Progettazione dei test

Definire come testare.

Comprende: creare casi di test basati sulle condizioni di test, assegnare priorità ai test in base a importanza e rischio, identificare dati di test e configurare l'ambiente di test,

mantenere tracciabilità bidirezionale tra base di test, condizioni e casi di test.
L'obiettivo è garantire che i test siano completi e ben organizzati.

5. Implementazione dei test

Preparare tutto per l'esecuzione.

Comprende: creare e dare priorità alle procedure di test, sviluppare script automatizzati, se necessario, creare e configurare l'ambiente di test (strumenti, simulatori, ecc.), preparare i dati di test e validare l'ambiente, mantenere la tracciabilità tra tutti gli elementi del processo di testing.

L'obiettivo è avere tutto pronto per l'esecuzione dei test.

6. Esecuzione dei test

Eseguire i test, registrare i risultati e identificare difetti.

Comprende: registrare la versione del software testato, eseguire i test, manualmente o con strumenti automatizzati, confrontare risultati attesi ed effettivi, investigando le discrepanze, segnalare i difetti e registrare i risultati dei test, ripetere i test dopo correzioni o aggiustamenti.

L'obiettivo è identificare eventuali difetti e validare i risultati.

7. Completamento del test

Chiudere l'attività di test, raccogliendo i risultati e preservando la conoscenza.

Comprende: verificare che tutte le segnalazioni di difetti siano risolte o documentate, creare un report finale con risultati, copertura e rischi rimanenti, archiviare l'ambiente di test e i dati per un possibile riutilizzo futuro, analizzare le lezioni apprese per migliorare il processo di test.

L'obiettivo è assicurarsi che nessuna informazione critica venga persa e migliorare il processo di testing per il futuro.

9) Cosa si intende per review? Come si progettano le review? Quali sono i tipi di review?

La review è il processo di analisi che non valuta unicamente il codice sorgente, ma anche tutti gli artefatti e i documenti prodotti (diagrammi UML, guide, modelli di design e UI).

Si distinguono in review informali, che non seguono specifiche regole e spesso sono consigli chiesti ad hoc basati su condivisione di conoscenze; review formali, con strutture rigide, ruoli definiti e risultati documentati, tendenzialmente usati in contesti ampi e maturi.

Il processo che si segue per progettare una review si articola in modo ciclico in queste fasi:

Planning, che ne definisce lo scopo, stima l'effort e definisce i partecipanti, ruoli e compiti

Inizializzazione, che condivide documenti e artefatti, chiarificare processi e risultati attesi

Revisione Individuale, in cui ciascun membro del team studia il sistema e individua errori

Comunicazione e Analisi, che avviene tipicamente tramite meeting e log di difetto

Fixing e Report, in cui gli autori correggono gli errori e verificano criteri e metriche richieste

10) Che cosa si intende per black/white box testing (equivalence partitioning, boundary value analysis)

Tecniche black-box: derivano i casi di test direttamente da una specifica o da un modello di un sistema o di un sistema proposto. Si basano su un'analisi della documentazione della base di test, inclusi gli aspetti funzionali e non funzionali. Non utilizzano alcuna informazione relativa alla struttura interna del componente o del sistema sottoposto a test.

Tecniche white-box: derivano i casi di test direttamente dalla struttura di un componente o sistema. Si concentrano sui test basati sul codice scritto per implementare un componente o un sistema.

La tecnica di test senza alcuna conoscenza del funzionamento interno dell'applicazione è chiamata test black-box. In genere, durante l'esecuzione di un test black-box, un tester interagisce con l'interfaccia utente del sistema fornendo input ed esaminando gli output senza sapere come e dove vengono elaborati gli input.

Advantages	Disadvantages
Well suited and efficient for large code segments.	Limited coverage, since only a selected number of test scenarios is actually performed.
Code access is not required.	Inefficient testing, due to the fact that the tester only has limited knowledge about an application.
Clearly separates user's perspective from the developer's perspective through visibly defined roles.	Blind coverage, since the tester cannot target specific code segments or error-prone areas.
Large numbers of moderately skilled testers can test the application with no knowledge of implementation, programming language, or operating systems.	The test cases are difficult to design.

Esistono 4 modi per fare il black-box test:

1. Partizionamento di equivalenza

Il partizionamento delle classi di equivalenza è una tecnica basata sulle specifiche (ovvero, una tecnica black-box).

Si basa sulla suddivisione dell'input in diverse classi equivalenti allo scopo di individuare gli errori.

Le classi di equivalenza copriranno l'intero dominio di input. In nessun punto una classe si sovrappone ad un'altra.

- classi di equivalenza valide: descrivono situazioni che il sistema dovrebbe gestire normalmente.
- classi di equivalenza non valide: descrivono situazioni non valide che il sistema dovrebbe rifiutare.

→ Creiamo un insieme di test validi con un membro valido da ogni partizione di equivalenza. Continuiamo questo processo finché ogni classe valida non è rappresentata in almeno un test valido.

→ Successivamente, possiamo creare test non validi.

Selezioniamo un membro non valido per una partizione di equivalenza e un membro valido per ogni altra partizione. Non combinare più membri non validi in un singolo test.

Continuiamo il processo finché ogni classe non è rappresentata in almeno un test non valido.

Ogni classe, sia valida che non valida, è rappresentata in almeno un test.

- Le partizioni di equivalenza devono essere disgiunte. Cioè, due sottoinsiemi non possono avere uno o più elementi in comune.
- Nessuna delle partizioni di equivalenza può essere vuota.

→ **MANCA UN ESEMPIO**

2. Analisi dei valori limite;

L'analisi dei valori al contorno utilizza le stesse classi del partizionamento di equivalenza, eseguendo il test ai confini anziché su un elemento della classe.

Un'analisi completa dei valori al contorno richiede il test del confine, del caso immediato all'interno del confine e del caso immediato all'esterno del confine.

Questa tecnica può generare un numero elevato di casi di test.

Casi minimi da fare:

- valore estremo inferiore (incluso)
- valore estremo superiore (incluso)
- valore mediano

- valore estremo inferiore - 1
- valore estremo superiore +1

→ fare attenzione a che tipo di dato sto utilizzando (double, int...)

→ **MANCA UN ESEMPIO**

3. Test della tabella delle decisioni;
4. Test dei casi d'uso.

Il test white-box è l'analisi dettagliata della logica interna e della struttura del codice.

Il test white-box è anche chiamato glass testing o open-box testing. Per eseguire il test white-box su un'applicazione, un tester deve conoscere il funzionamento interno del codice.

Il tester deve dare un'occhiata all'interno del codice sorgente e scoprire quale unità/chunk del codice si comporta in modo inappropriato.

Advantages	Disadvantages
As the tester has knowledge of the source code, it becomes very easy to find out which type of data can help in testing the application effectively.	Due to the fact that a skilled tester is needed to perform white-box testing, the costs are increased.
It helps in optimizing the code.	Sometimes it is impossible to look into every nook and corner to find out hidden errors that may create problems, as many paths will go untested.
Extra lines of code can be removed which can bring in hidden defects.	It is difficult to maintain white-box testing, as it requires specialized tools like code analyzers and debugging tools.
Due to the tester's knowledge about the code, maximum coverage is attained during test scenario writing	

11) Che cosa sono e a cosa servono le decision table?

Le tabelle decisionali ci permettono di testare il modo in cui diverse combinazioni di condizioni interagiscono per produrre determinati risultati che si verificano e non si verificano.

Abbiamo almeno un test per ogni colonna.

Le tabelle decisionali non sono in genere utili per i test unitari (troppo semplici).

Grazie alla loro struttura concisa, consentono di progettare test per tali regole, di solito almeno un test per regola.

Conditions	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Real account?	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N
Active account?	Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N
Within limit?	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Location okay?	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N
Actions																
Approve?		Y	N	N	N	N	N	N	N	N	N	N	N	N	N	N
Call cardholder?	N	Y	Y	Y	N	Y	Y	Y	N	N	N	N	N	N	N	N
Call vendor?	N	N	N	N	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

Se ci sono casistiche uguali la tabella può collassare. (esempio casi 9-16)

Conditions	1	2	3	5	6	7	9
Real account?	Y	Y	Y	Y	Y	Y	N
Active account?	Y	Y	Y	N	N	N	-
Within limit?	Y	Y	N	Y	Y	N	-
Location okay?	Y	N	-	Y	N	-	-
Actions							
Approve?	Y	N	N	N	N	N	N
Call cardholder?	N	Y	Y	N	Y	Y	N
Call vendor?	N	N	N	Y	Y	Y	Y

12) Che cosa sono e a che cosa servono i grafici causa/effetto

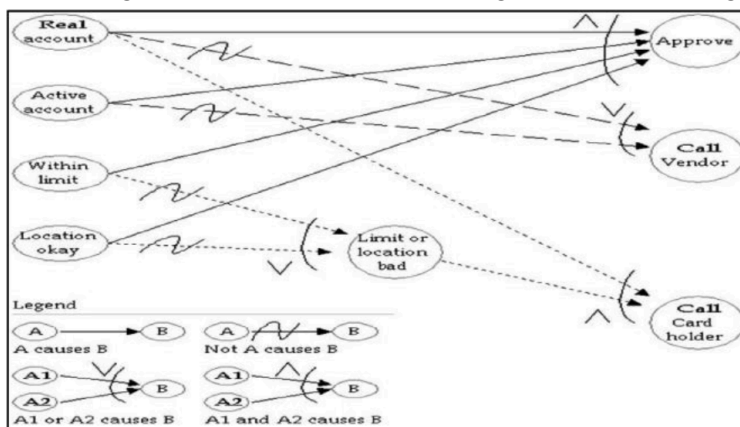
I grafici causa-effetto sono rappresentazioni grafiche della stessa logica aziendale mostrata in una tabella decisionale.

È sempre possibile convertirli da uno all'altro.

Si applicano allo stesso modo e alle stesse situazioni di una tabella decisionale.

Per creare i grafici sono necessari degli strumenti e bisogna imparare la notazione.

I test sui grafici causa-effetto rilevano gli stessi tipi di bug delle tabelle decisionali.



13) E' possibile generare test a partire da Use Case?

Si è possibile generare test a partire da use case.

È un modo per garantire di aver testato flussi di lavoro e scenari tipici ed eccezionali per il sistema, dal punto di vista dei vari attori che interagiscono direttamente con il sistema e dal punto di vista dei vari stakeholder che interagiscono indirettamente con il sistema.

È importante che il sistema sia sufficientemente completo per l'utilizzo previsto. L'utilizzo realistico del sistema, inclusi gli utilizzi atipici, deve essere ben compreso, altrimenti si verificheranno lacune nei test. I test devono coprire l'utilizzo tipico, il percorso principale o percorso positivo, nonché gli utilizzi atipici, chiamati percorsi alternativi o percorsi di eccezione.

Questa tecnica può convalidare il software verificando l'elaborazione non corretta dei percorsi principali o alternativi, la mancanza di percorsi alternativi e problemi con la segnalazione degli errori.

Per derivare i test dai casi d'uso:

Dovremmo creare un test per ogni flusso di lavoro, inclusi sia i flussi di lavoro tipici che quelli eccezionali.

Se i flussi di lavoro eccezionali sono stati omessi, sarà necessario ricavarli dai requisiti o da altre fonti.

Non testare le eccezioni è un errore di test comune quando si utilizzano i casi d'uso.

La creazione di test può comportare l'applicazione del partizionamento di equivalenza e dell'analisi dei valori limite.

Possiamo facilmente tradurre un caso d'uso in uno o più test logici. La traduzione dei test logici in test concreti può richiedere ulteriore documentazione.

14) Cosa si intende per control flow graph

(Tecnica del white-box test)

Un approccio al test basato sulla struttura in cui i casi di test sono progettati per eseguire specifiche sequenze di eventi.

Il test del flusso di controllo viene eseguito tramite grafici di flusso di controllo, che forniscono un modo per astrarre un modulo di codice per comprenderne meglio le funzioni.

I grafici di flusso di controllo forniscono una rappresentazione visiva della struttura del codice.

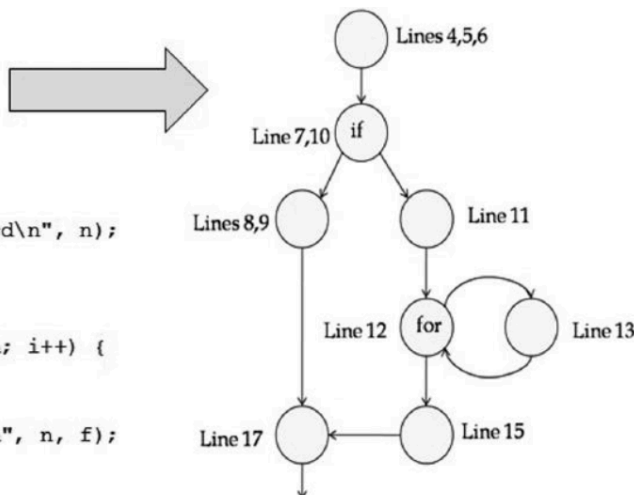
Ci permettono di analizzare i possibili percorsi attraverso il grafico.

Ogni possibile percorso → test da fare

(Copertura delle istruzioni; Copertura delle decisioni (o diramazioni); Copertura delle condizioni / copertura di condizioni multiple).

I punti di decisione ci costringono a effettuare più test, almeno un test per ogni percorso, per prendere la decisione in modo diverso, modificando il modo in cui attraversiamo il codice.

```
1 #include <stdio.h>
2 main()
3 {
4     int i, n, f;
5     printf("n = ");
6     scanf("%d", &n);
7     if (n < 0) {
8         printf("Invalid: %d\n", n);
9         n = -1;
10    } else {
11        f = 1;
12        for (i = 1; i <= n; i++) {
13            f *= i;
14        }
15        printf("%d! = %d\n", n, f);
16    }
17    return n;
18 }
```



15) Che cosa si intende per **statement coverage**, **decision coverage**, **loop coverage**, **condition coverage** e **modified condition/decision coverage**

La **Statement Coverage** (copertura delle istruzioni) è una misura della copertura del codice in base al numero di istruzioni che sono state eseguite almeno una volta durante il test. Se una dichiarazione nel codice viene eseguita almeno una volta, viene considerata "coperta". In sostanza, misura se ogni riga di codice è stata visitata almeno una volta durante l'esecuzione dei test. → Se il tuo codice contiene una funzione con 10 istruzioni e i tuoi test coprono 8 di esse, la tua statement coverage sarà 80%.

La **Decision Coverage** (copertura delle decisioni) si concentra sulle espressioni logiche che conducono a una decisione (come un'istruzione `if`, `switch` ecc.). In altre parole, misura se ogni espressione booleana nel programma è stata valutata come **vera** e **falsa** almeno una volta. Dobbiamo quindi verificare che tutte le decisioni logiche (come le condizioni `if`) siano state esaminate in entrambe le direzioni (vero e falso). → Nel caso di una condizione `if (a > 5)`, il test deve coprire sia il caso in cui `a > 5` (vero) che quello in cui `a <= 5` (falso).

La **Loop Coverage** (copertura dei cicli) si riferisce alla verifica del comportamento del programma all'interno dei cicli (ad esempio `for`, `while`). Misura se ogni ciclo nel programma è stato eseguito almeno una volta, e se sono stati testati sia i casi in cui il ciclo non viene eseguito che quelli in cui viene eseguito. Dobbiamo quindi verificare che i cicli siano stati attraversati in vari modi: mai, una volta o più volte. Per farlo possiamo usare i valori che usavamo per i casi di test (estremo superiore, inferiore e un valore a metà). → Per un ciclo `for (int i = 0; i < 10; i++)`, dovresti testare il caso in cui il ciclo non viene eseguito (ad esempio, se `i` è inizializzato a un valore maggiore di 10) e i casi in cui il ciclo viene eseguito almeno una volta.

Importante: il decision coverage implica lo statement coverage ma non vale il contrario.

La **Condition Coverage** (copertura delle condizioni) misura se ogni singola condizione booleana all'interno di una decisione (ad esempio, in un `if`) è stata testata in entrambe le direzioni (vero e falso). Questo si concentra sulle singole condizioni all'interno di espressioni complesse. Dobbiamo quindi verificare che ogni condizione booleana in una decisione (come un `if` o una combinazione di condizioni) sia stata testata come **vera** e **falsa**. → In un'espressione del tipo `if (a > 5 && b < 10)`, la condition coverage richiede che tu testi `a > 5` come vero e falso, e `b < 10` come vero e falso.

La **Modified Condition/Decision Coverage (MC/DC)** è una forma avanzata di copertura che estende la condition coverage e la decision coverage. Essa richiede che ogni condizione all'interno di una decisione sia testata per determinare se ha un effetto indipendente sulla decisione complessiva. Dobbiamo verificare che ogni condizione booleana in una decisione influenzi la decisione stessa. Ogni condizione deve essere testata separatamente in modo da dimostrare che ha un impatto indipendente sulla decisione finale. → Nel caso di `if (a > 5 && b < 10)`, dovrai testare almeno una volta ciascuna condizione (`a > 5` e `b < 10`) in modo che il cambiamento di una condizione possa influenzare il risultato della decisione finale.

16) Semplici esercizi sul white testing: può essere richiesta l'individuazione di test da fare per raggiungere la massima copertura di statement o decision.

17) Che cosa è il path testing

Si tratta di una particolare tecnica di testing white-box che analizza i percorsi che può intraprendere il software, preferendo sempre i più corti, semplici e sensati, evitando le iterazioni. Secondo la teoria di McCabe esistono dei percorsi base che combinati in vari modi permettono di raggiungere una copertura totale del codice. Il processo di path testing è articolato nella fase di analisi del numero di percorsi, inclusione all'interno dei test di tali percorsi, testing di quelli linearmente indipendenti (L.I.). I percorsi logici sono 2^n (con n numero di decisioni) mentre il numero di percorsi L.I. corrispondono al numero cicломatico aumentato di 1.

18) Altre tecniche di test: ispezioni e review.

Le review sono utilizzate come strumento per controllo aggiuntivo sul software: i diversi tipi di review vengono suddivise in base al livello di formalità, dalla meno formale alla più.

Per sfruttare al meglio le review, è necessario pianificare con cura:

- la porzione di sistema interessata alla review
- suddivisione del lavoro di controllo
- persone coinvolte nella review
- tempo e denaro investito nella review
- preparazione eventuale di documenti

Una volta completata la review, è essenziale che siano presentati i risultati utilizzando metriche che siano rilevanti ai fini del problema. La comunicazione prima, durante e dopo la review è necessaria per coordinare ed evitare problemi di discrepanze future, oltre che perdita di tempo nel processo.

Ruoli coinvolti:

- moderator: mediatore per risolvere conflitti e prendere la decisione finale di rilasciare i documenti aggiornati
- leader: la persona indicata per condurre tutto il processo. Deve assicurare la coordinazione e il raggiungimento degli obiettivi nei tempi previsti.
- reviewers: sono i tecnici che si occuperanno della risoluzione dal punto di vista tecnico.
- scribe: si occupa di documentare tutte le riunioni e i cambiamenti
- author: responsabile dei documenti di review, sarà la persona a risolvere il problema individuato collettivamente.
- manager: normalmente non partecipa ma agisce sotto il punto di vista organizzativo (su cosa sarà la review, tempo e costo assegnati...)

Tipi di review (dalla informale alla più formale):

- Informale: estremamente ad hoc, generalmente usata per brainstorming o risolvere disambiguità o problemi di piccola entità. Il processo e la documentazione variano molto da review a review

- Walkthrough: usate soprattutto per migliorare il software o renderlo conforme a standard esistenti. Il livello di formalità può variare molto, ma è necessario lo scribe per documentare decisioni e cambiamenti avvenuti. Si usano delle checklist per avere una scaletta di discussione. Molto spesso sono indicate per un gruppo specifico di persone assegnate a una particolare task.
- Technical review: si cerca di raggiungere il consenso comune e di individuare eventuali problemi nel software. E' necessario lo scribe per la documentazione, e sarebbe meglio che questo NON sia l'autor. La preparazione pre-review è necessaria e sono inserite nella review persone specializzate nell'argomento interessato.
- Inspection: massima formalità. L'obiettivo è quello di prevenire bug futuri e risolvere gli attuali aumentando l'affidabilità nel software. Richiede una riunione apposita, coordinata dal moderator. Si usano tutti gli strumenti di misurazione di metriche e sono decisi criteri di ingresso/uscita formali dalla review. La preparazione pre-meeting è essenziale data la alta formalità della review. Tutto è documentato dallo scribe.

19) Tecniche automatizzate: perché sono importanti.

Le tecniche di test automatizzati sono fondamentali perché garantiscono efficienza, affidabilità e velocità nel ciclo di sviluppo. Rispetto ai test manuali, l'automazione permette di eseguire migliaia di verifiche in pochi minuti, facilitando la Continuous Integration e la Continuous Delivery. Assicura che le nuove modifiche non introducono bug (regression testing). Tramite i test automatizzati è effettuata l'analisi statica del codice (coverage e struttura del codice).

✓ Configuration Management, Integration, and Builds → COMPLETA

1) Cosa si intende per Software configuration management? Da che cosa dipendono gli strumenti software utilizzati a supporto di questa fase?

Il software configuration management è il processo che serve a gestire tutti i singoli pezzi e le parti degli artefatti prodotti durante lo sviluppo e il supporto del software. Questo processo consiste nel comprendere le attività e i risultati da gestire, definire il framework di gestione e assicurarsi che il personale sia addestrato a seguire queste procedure. Gli strumenti utilizzati e il livello di dettaglio della gestione dipendono direttamente dal processo di sviluppo scelto dall'organizzazione. Ad esempio, un'azienda che segue un approccio conservativo per software complessi potrebbe gestire in modo meticoloso ogni versione di requisiti, design, codice sorgente, eseguibili e casi di test. Al contrario, un'organizzazione piccola e agile potrebbe limitarsi a controllare solo le versioni finali dei requisiti e del codice. In sostanza, è la politica aziendale a determinare quali elementi devono essere tracciati e con quale rigore.

2) Quali sono le funzioni fondamentali che supportano lo storage e la fase di integrazione e build ?

Le funzioni fondamentali per lo storage e l'accesso agli artefatti includono operazioni di base come creare (create), eliminare(delete), visualizzare(view), modifica (modify) e ritorno(return). Esistono funzioni aggiuntive come l'importazione (check-in) e l'esportazione (check-out), la promozione degli artefatti verso nuovi stati (ad esempio da sviluppo a test) e il confronto (compare) tra versioni diverse. Molto importanti sono anche i meccanismi di blocco (locking), per impedire a più utenti di modificare lo stesso file contemporaneamente, e la funzione "where-referenced", che permette di capire quali documenti o moduli dipendono da un elemento che sta per essere cambiato.

Per la fase di integrazione e build, il processo trasforma i file sorgente in componenti eseguibili pronti per l'uso. Questo avviene principalmente attraverso due fasi: la compilazione, che traduce ogni file sorgente in un formato leggibile dalla macchina, e il collegamento (link), che unisce i moduli risolvendo i riferimenti a librerie esterne. Tutte queste attività sono gestite dai descriptor file (ad esempio i makefile) che controllano l'ordine corretto delle operazioni e verificano le dipendenze. Per risparmiare tempo in sistemi di grandi dimensioni, si utilizzano spesso le "build incrementali" (Incremental Builds), che ricompilano e collegano solo le parti del software che sono state effettivamente modificate.

✓ **Software Support and Maintenance → COMPLETA**

1) Perché la fase di supporto del software e manutenzione è importante? Quali sono le operazioni che vengono svolte in questa fase?

Il supporto e la manutenzione del software rappresentano una fase cruciale poiché il periodo post-rilascio è diverse volte più lungo rispetto a quello di sviluppo.

La vitalità a lungo termine e il successo di un prodotto dipendono molto dall'esperienza reale dell'utente, dato che software complessi possono contenere difetti anche dopo test estesi o presentare mancanze dovute a rilasci incrementali forzati da vincoli di tempo.

In questa fase, le operazioni si dividono principalmente in supporto per difetti (Product Defect Support), che include diagnosi, prioritizzazione e risoluzione tramite patch o hotfix, e supporto non-difettoso (Product Non-Defect Support and Services), che riguarda l'assistenza nell'installazione, la formazione dell'utente e la gestione di richieste di miglioramento.

2) E' importante stimare il tasso di notifica di problemi?

È essenziale stimare il tasso di arrivo dei problemi per pianificare le risorse di manutenzione, comprendere la qualità del prodotto e prevedere le necessità di supporto a lungo termine.

Un basso numero iniziale di segnalazioni non indica necessariamente un'alta qualità, poiché deve essere sempre rapportato al tasso di utilizzo reale del software.

3) In che modo può essere strutturato il centro di supporto? Quali sono i ruoli che svolge?

L'organizzazione del centro di supporto è solitamente strutturata su più livelli per garantire efficienza. Il primo livello è costituito dai rappresentanti del servizio clienti che gestiscono l'identificazione dell'utente, registrano la descrizione del problema e cercano soluzioni immediate tramite FAQ o knowledge base, risolvendo i casi che non richiedono modifiche al codice. Quando un problema è troppo complesso, viene scalato al secondo livello composto da analisti tecnici altamente qualificati. Questi esperti hanno il compito di analizzare e riprodurre il problema per poi avviare un ciclo di design, codifica e test finalizzato alla creazione di un Change Request (CR) e di una correzione permanente.

4) Quali sono i problemi connessi con la distribuzione e l'installazione delle patch di manutenzione / miglioria del software?

La distribuzione e l'installazione delle patch presentano diverse problematiche, a partire dall'ordine dei rilasci periodici che i clienti dovrebbero installare in ordine cronologico. Molti clienti non applicano nuovi fix per timore di compromettere un sistema già funzionante, ma questo rende le applicazioni retroattive estremamente dispendiose in futuro. Inoltre, i fix di emergenza applicati immediatamente possono entrare in conflitto con i successivi rilasci regolari, obbligando talvolta il cliente a rimuovere la correzione temporanea prima di procedere con l'aggiornamento completo. Infine, la necessità di coordinare patch "co-requisite" tra software diversi (come sistemi operativi e database) e il rischio di regressioni dovute a una scarsa disciplina ingegneristica nella documentazione dei piccoli fix complicano ulteriormente la gestione della manutenzione.